



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**



Grado en Matemáticas e Informática

Trabajo Fin de Grado

**Diseño y desarrollo de un sistema
Multi-Tenant de Altas de Empleados
conforme al RGPD y la API de la
Seguridad Social (TGSS) con tecnología
Low-Code**

Autor: Alejandro García Ibáñez

Tutor: Elena Villalba Mora

Madrid, Junio - 2026

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado

Grado en Matemáticas e Informática

Título: Diseño y desarrollo de un sistema Multi-Tenant de Altas de Empleados conforme al RGPD y la API de la Seguridad Social (TGSS) con tecnología Low-Code

Junio - 2026

Autor: Alejandro García Ibáñez

Tutor: Elena Villalba Mora

Departamento de Lenguajes y Sistemas Informáticos e Ingeniería de Software
Escuela Técnica Superior de Ingenieros Informáticos
Universidad Politécnica de Madrid

Agradecimientos (Revisar)

A todos los profesores del grado, por ayudarme en mi formación académica en las área de matemáticas e informática, labor por la cual les estoy eternamente agradecido.

A Elena, mi tutora, porque a pesar de no haber sido mi profesora, siempre ha estado dispuesta a echarme una mano con lo que hiciera falta.

A los profesores del Colegio Logos International School, con especial mención a Maria José y Pilar García, profesores que creyeron en mi potencial y fueron un paso más allá para proponerme retos para mejorar como alumno.

A mis amigos, con los que he compartido muchos momentos de alegría, de tristeza, y que me han ayudado a crecer como persona.

A mis padres, por la educación recibida, que me han hecho ser quien soy.

A mis hermanos: Álvaro, por todas las noches que nos quedamos hablando y las muchas tonterías que hacemos juntos; Laura, quien siempre me ha escuchado sin juzgarme; Claudia, con la que me peleó muchas mañanas para llegar puntual al colegio; y por último, Alberto, mi referente, la persona a la que aspiro ser, y el mejor hermano mayor que un hermano pequeño puede pedir.

Resumen

En España, el marco legal exige a las empresas tramitar el alta de sus trabajadores en la Seguridad Social de manera previa al inicio de su actividad laboral. Este proceso se comunica habitualmente a través del Sistema RED y herramientas de escritorio específicas como SILTRA [seguridadsocial2026]. No obstante, la interfaz actual de la administración presenta una notable complejidad técnica y una experiencia de usuario poco intuitiva, lo que obliga a las gestorías a realizar tareas manuales propensas a errores. La ausencia de una API REST pública por parte de la Tesorería General de la Seguridad Social (TGSS) agrava esta carga operativa al requerir una gestión fragmentada de certificados digitales.

Este Trabajo de Fin de Grado (TFG) presenta el diseño y desarrollo de una plataforma web y móvil orientada a gestorías de Recursos Humanos que automatiza, digitaliza y simplifica el proceso de alta. El sistema se centra en la optimización de la experiencia de usuario (UX), sustituyendo la opacidad de los formularios oficiales por una interfaz comprensible que delega la complejidad de los cálculos y reglas de negocio a procesos de *background* o segundo plano. Para ello, se ha diseñado una arquitectura *Multi-Tenant* que garantiza la segregación estricta de la información y el cumplimiento del Reglamento General de Protección de Datos (RGPD)[rgpd2016].

A nivel tecnológico, se ha empleado el paradigma *Low-Code* [lowcode2021richardson] mediante FlutterFlow y el ecosistema Firebase (Firestore y Cloud Functions). Tras la validación algorítmica de los datos en tiempo real, el sistema asegura la integridad de la información antes de su remisión. Ante la falta de una pasarela directa con la administración, el flujo de datos culmina en el envío de la información verificada hacia un *endpoint* facilitado por la empresa, el cual actúa como nodo de comunicación final hacia la TGSS. Los resultados muestran un prototipo funcional robusto que reduce la carga cognitiva del operador y ofrece un entorno seguro, escalable y eficiente.

Palabras Clave: Multi-Tenant, FlutterFlow, Firebase, UX, Abstracción de Procesos, Seguridad Social, RGPD, Low-Code.

Abstract

In Spain, the legal framework requires companies to register employees with the Social Security system prior to their labor activity. This procedure is typically managed through the "Sistema RED," and specific desktop tools like SILTRA. However, the current administrative interface is characterized by high technical complexity and a non-intuitive user experience, forcing HR agencies into error-prone manual tasks. The absence of a public REST API from the General Treasury of the Social Security (TGSS) further increases this burden due to fragmented digital certificate management.

This Final Degree Project (TFG) presents the design and development of a web and mobile platform for HR agencies that automates, digitizes, and simplifies the registration process. The system focuses on User Experience (UX) optimization, replacing the opacity of official forms with a clear interface that delegates complex calculations and business rules to background processes. To support this, a Multi-Tenant architecture was designed to ensure strict data segregation and compliance with the General Data Protection Regulation (GDPR).

Technologically, the project leverages the Low-Code paradigm using FlutterFlow and the Firebase ecosystem (Firestore and Cloud Functions). Following real-time algorithmic validation to ensure data integrity, and given the lack of a direct gateway to the administration, the data flow concludes by transmitting the verified information to a company-provided endpoint, which serves as the final communication node with the TGSS. The results demonstrate a robust functional prototype that reduces the operator's cognitive load and provides a secure, scalable, and efficient environment.

Keywords: Multi-Tenant, FlutterFlow, Firebase, UX, Process Abstraction, Social Security, GDPR, Low-Code.

Índice general

1. Introducción	1
1.1. Motivación del proyecto	1
1.2. Contexto del proyecto	1
1.3. Objetivos	2
1.4. Estructura del Documento (revisar)	3
2. Trabajo relacionado y Estado del Arte (revisar)	4
2.1. El Sistema RED y la comunicación oficial con la TGSS	4
2.2. Soluciones alternativas y APIs de terceros	5
2.3. Tecnologías Low-Code y Backend-as-a-Service (BaaS)	6
2.4. Arquitecturas Multi-Tenant y Cumplimiento del RGPD	6
3. Metodología y Herramientas	8
3.1. Metodología de Desarrollo	8
3.2. Herramientas Empleadas	9
3.2.1. Herramientas de Colaboración y Gestión Laboral	9
3.2.1.1. Microsoft Teams	9
3.2.1.2. Microsoft Word	9
3.2.1.3. Microsoft PowerPoint	9
3.2.1.4. Microsoft Excel	9
3.2.1.5. Microsoft Copilot	9
3.2.2. Herramientas de Desarrollo Frontend	10
3.2.2.1. FlutterFlow	10
3.2.2.2. Dart	10
3.2.2.3. Material Design y Heurísticas de Nielsen	10
3.2.3. Herramientas de Desarrollo Backend e Infraestructura	11
3.2.3.1. Firebase	11
3.2.3.2. Firestore	11
3.2.3.3. Firebase Authentication	11
3.2.4. Herramientas de Apoyo y Control de Versiones	11
3.2.4.1. GitHub	11
3.2.4.2. Script de Automatización	12
3.2.5. Herramientas de Redacción y Formato	12
3.2.5.1. Visual Studio Code	12
3.2.5.2. LaTeX	12
4. Módulo de Gestión	13
4.1. Gestión de Usuarios y Autenticación	13

4.1.1. Proceso de Login y Peticiones a Firebase	13
4.1.1.1. Uso de Firebase Authentication	13
4.1.1.2. Tipos de Usuarios	13
4.1.2. Delegación de Permisos	14
4.2. Estructura del Frontend	14
4.3. Estructura del Backend	14
5. Módulo de Autenticación y Control de Roles	16
5.1. Pantalla Principal y Opciones de Usuario	16
5.1.1. Opciones de Administrador	17
5.1.2. Opciones de Supervisor	17
5.1.3. Opciones del Usuario de Recursos Humanos	18
5.2. Gestión de Datos y Formulario Dual	18
5.2.1. Almacenamiento en Google Auth	18
5.2.2. Almacenamiento en Firestore	20
5.2.2.1. Campos de Identificación y Roles	21
5.2.2.2. Control de Acceso Multi-Tenant	22
5.2.2.3. Lista de datos de un usuario	22
5.3. Cumplimiento del RGPD y Asignación Manual	23
6. Módulo de Gestión de Empresas	24
6.1. Operativas de Gestión	24
6.1.1. Creación de Nueva Empresa	24
6.1.2. Edición y Gestión de Empresas Existentes	25
6.2. Datos del Formulario de Empresa	26
6.2.1. Identificación y Datos de Registro	26
6.2.1.1. Identificación Corporativa	26
6.2.1.2. Información de Domicilio	26
6.2.2. Parámetros de Cotización y Actividad (TGSS)	27
6.2.2.1. Datos de Afiliación a la Seguridad Social	27
6.2.2.2. Actividad y Fiscalidad	27
6.2.3. Datos de Contratos y Convenios	27
6.2.3.1. Convenio Aplicable	27
6.2.3.2. Estructura Salarial	27
6.2.4. Datos de Firma y Documentación	28
6.2.4.1. Firma y Representación Legal	28
6.2.4.2. Gestión Documental y Privacidad	28
7. Módulo de Gestión de Empleados	29
7.1. Operativa de Empleados: Alta y Edición	29
7.1.1. Proceso de Registro y Permisos (revisar)	29
7.1.2. Edición de Datos y Trazabilidad	30
7.2. Datos del Formulario de Empleados	30
7.2.1. Datos de Identificación y Contacto	31
7.2.1.1. Identificación Oficial y Afiliación	31
7.2.1.2. Filiación y Datos Personales	31
7.2.1.3. Información de Contacto	31
7.2.2. Datos de Domicilio	31
7.2.2.1. Dirección Física y Geográfica	31
7.2.3. Datos de Contrato y Formación	32

7.2.3.1. Vinculación Corporativa y Académica	32
7.2.3.2. Condiciones y Datos Bancarios	32
7.2.4. Constantes (Uso Futuro y Lógica de Contrato)	32
7.2.4.1. Parámetros Oficiales de Seguridad Social	32
7.2.4.2. Condiciones Laborales y Retributivas	33
7.2.4.3. Temporalidad, Antigüedad y Periodo de Prueba	33
8. Pruebas	34
8.1. Pruebas de seguridad	34
8.1.1. Elevación de privilegios	34
8.1.2. Path Traversal	35
9. Impacto del trabajo	36
9.1. Impacto general	36
9.2. Objetivos de Desarrollo Sostenible	37
10.Resultados y conclusiones	38
10.1 Resultados	38
10.2.Conclusiones personales	39
10.3.Trabajo futuro	39
11.Anexo	41
11.1 Estructuras de dades	41
11.1.1 Lista de datos de una empresa corporativa	41
11.1.2 Lista de datos de un empleado	43
11.1.3 Lista de datos de un convenio	45
11.1.4 Lista de datos de un municipio	46
11.2 Funciones de Dart utilizadas en el desarrollo	46
11.2.1 Funciones de validación de campos	46
11.2.1.1 Validación de DNI/NIE	46
11.2.1.2 Limitación de la edad a un mínimo de 16 años	47
11.2.1.3 Limitación de la fecha de alta a un máximo de 60 días posteriores a la fecha actual	48
11.2.1.4 Validar NASS	48
11.2.1.5 Enviar alta	48
11.2.1.6 Obtener referencias de documentos a partir de sus IDs	49
11.2.1.7 Obtener referencia de un usuario	49
11.2.1.8 Crear un nuevo usuario en la colección de usuarios	49
11.2.2 Devolver una referencia de usuario concreta	50
11.2.2.1 Obtener múltiples referencias de empresas	51
11.2.2.2 Enviar los datos de empleados a los endpoints de la em- presa	51

Capítulo 1

Introducción

Este primer capítulo presenta una visión general del Trabajo de Fin de Grado. En la primera sección se expone la motivación y necesidad de abordar la problemática actual en las gestorías laborales. A continuación, se detalla el contexto tecnológico y normativo del proyecto. Posteriormente, se define el objetivo principal y se desglosa en una serie de objetivos específicos técnicos. Finalmente, se describe la estructura del resto del documento para facilitar su lectura.

1.1. Motivación del proyecto

En España, la normativa legal obliga a las empresas a solicitar el alta de sus trabajadores en el régimen de la Seguridad Social de manera previa al inicio real de la prestación de los servicios. En la actualidad, este proceso resulta ser altamente repetitivo y dependiente de herramientas de escritorio proporcionadas por la administración.

La motivación principal de llevar a cabo este Trabajo de Fin de Grado es desarrollar una aplicación web/móvil destinada a automatizar y simplificar de manera drástica el proceso de alta de empleados en la Seguridad Social por parte de las gestorías. Al gestionar múltiples clientes, los despachos laborales sufren una importante carga administrativa y propensión a cometer errores en la introducción de datos. Por consiguiente, existe la necesidad imperante de crear una herramienta moderna que optimice este flujo de trabajo, la cual deberá garantizar la integridad de los datos, la trazabilidad exhaustiva de cada alta procesada y el cumplimiento estricto de la normativa europea de protección de datos (RGPD).

1.2. Contexto del proyecto

El contexto tecnológico de las comunicaciones obligatorias con la Tesorería General de la Seguridad Social (TGSS) en España gira en torno al Sistema RED (Remisión Electrónica de Datos). Este sistema oficial exige el uso de ficheros muy específicos y la gestión local de certificados digitales, careciendo de una API web pública, directa y sencilla basada en estándares modernos (como REST/JSON). Esta barrera tecnológica impide una integración ágil desde plataformas en la nube.

Para sortear este obstáculo y aportar una solución, este proyecto se contextualiza en el uso de tecnologías de vanguardia bajo el paradigma *Low-Code*. Se propone la utilización de **FlutterFlow** para el desarrollo de la interfaz frontend multiplataforma, y de **Firebase** como infraestructura backend (*Backend-as-a-Service*). Dado que el sistema debe dar servicio a gestorías que manejan múltiples empresas, el desarrollo se enmarca en una arquitectura *Multi-Tenant* (multi-inquilino), utilizando la base de datos NoSQL **Firestore** para garantizar la segregación absoluta y segura de los datos por cada empresa.

Además, debido a la imposibilidad técnica de interactuar directamente con el Sistema RED sin un complejo *software* intermediario, el proyecto contempla en su contexto de desarrollo la implementación de una *Mock API* (API simulada) mediante Cloud Functions, la cual actuará verificando y transformando los datos, así como simulando las respuestas de la administración.

1.3. Objetivos

El objetivo principal de este Trabajo de Fin de Grado es diseñar y desarrollar una plataforma web o móvil mediante tecnología *Low-Code* que automatice y simplifique de forma segura el proceso de alta de empleados en múltiples empresas, garantizando la integridad de los datos, la trazabilidad del proceso y cumpliendo estrictamente con el RGPD y la simulación de la API de la Seguridad Social (TGSS).

Para alcanzar este propósito general, el proyecto se divide en los siguientes objetivos específicos:

- **Implementar un modelo de seguridad Multi-Tenant:** Garantizar la segregación estricta de datos utilizando referencias a documentos (*Document Reference*) en Firestore, de forma que se soporte una gestión granular de permisos (administrador, RRHH, supervisor) cumpliendo con el RGPD.
- **Desarrollar lógica algorítmica para validaciones críticas:** Crear formularios de captura de datos (personales, contrato, jornadas) e implementar validaciones rigurosas en tiempo real en el *frontend* utilizando Dart (por ejemplo, validación de formato DNI/NIE y restricciones de fechas lógicas).
- **Diseñar y ejecutar Cloud Functions para la integración:** Programar funciones en la nube que actúen como *middleware* seguro para gestionar la transformación de datos, la gestión de tokens y la simulación de las llamadas a la API de la Seguridad Social externa.
- **Diseñar un sistema de Historial y Trazabilidad:** Implementar una colección histórica (*altas_history*) que permita el registro constante del estado de cada trámite (enviado, error, aceptado) y la visualización del historial segmentado por empresa y usuario.
- **Aplicar principios de experiencia de usuario (UX):** Desarrollar interfaces modernas y eficientes orientadas a gestorías, ofreciendo un entorno accesible que notifique adecuadamente alertas y mensajes de error en caso de datos inválidos.

1.4. Estructura del Documento (revisar)

Para facilitar la lectura y comprensión de este trabajo, y siguiendo las recomendaciones académicas para memorias de ingeniería, el resto del documento se ha estructurado de la siguiente forma:

- **Capítulo 2. Estado del Arte y Análisis de Requisitos:** En este capítulo se analiza la problemática vigente con el Sistema RED y se enumeran de manera exhaustiva los requerimientos funcionales y no funcionales del sistema propuesto.
- **Capítulo 3. Metodología y Tecnologías Empleadas:** Se detalla el enfoque de desarrollo utilizado y se justifican ampliamente las herramientas seleccionadas para el proyecto (*Flutterflow, Firebase, Firestore, Cloud Functions*).
- **Capítulo 4. Módulo de gestión de usuarios:** Detalla el núcleo del sistema enfocado en la administración, jerarquía y delegación de permisos de usuarios, así como la estructura frontend y backend asociada a la autenticación segura.
- **Capítulo 5. Módulo de Autenticación y Control de Roles:** Describe la arquitectura del sistema de accesos, detallando el almacenamiento dual entre Google Auth y Firestore, la segregación de permisos por roles y los mecanismos de control Multi-Tenant para el estricto cumplimiento del RGPD.
- **Capítulo 7. Impacto:** Se realiza un análisis crítico de los resultados obtenidos, valorando el grado de cumplimiento de los objetivos planteados y proponiendo posibles mejoras y líneas futuras de desarrollo.
- **Capítulo 8. Resultados:** Se incluyen capturas de pantalla, diagramas y fragmentos de código relevantes que ilustran el funcionamiento del sistema desarrollado.
- **Capítulo 9. Anexos:** Se adjuntan documentos complementarios como el código fuente completo, diagramas de arquitectura detallados, y cualquier otro material que aporte valor a la comprensión del proyecto.
- **Pruebas de seguridad:** Se describen las pruebas de seguridad realizadas para validar la robustez del sistema frente a vulnerabilidades comunes como elevación de privilegios y path traversal, detallando los métodos utilizados, los resultados obtenidos y las medidas implementadas para mitigar dichas vulnerabilidades.

Capítulo 2

Trabajo relacionado y Estado del Arte (revisar)

Este capítulo analiza el contexto tecnológico y normativo en el que se enmarca este Trabajo de Fin de Grado [**primera_entrega**] [1]. En primer lugar, se expone la situación actual de las comunicaciones con la Seguridad Social mediante el Sistema RED y la falta de interfaces modernas [**blog_diario**] [2, 3]. A continuación, se evalúan las soluciones alternativas disponibles en el mercado, como las APIs de terceros [**blog_diario**] [4, 5]. Posteriormente, se describe el estado del arte de las tecnologías de desarrollo ágil empleadas, específicamente el paradigma *Low-Code* y los servicios *Backend-as-a-Service* (BaaS) [**documento_diseno, proyecto_alta**] [6, 7]. Finalmente, se aborda la literatura relacionada con las arquitecturas *Multi-Tenant* y el estricto cumplimiento del Reglamento General de Protección de Datos (RGPD) [**primera_entrega, proyecto_alta**] [1, 8, 9].

2.1. El Sistema RED y la comunicación oficial con la TGSS

El Sistema RED (Remisión Electrónica de Datos) es el canal de comunicación obligatorio establecido por la administración pública española para que las empresas realicen trámites de afiliación, como altas, bajas y variaciones de datos, así como la cotización a la Seguridad Social [**blog_diario**] [2]. En el estado actual de la tecnología gubernamental, la Tesorería General de la Seguridad Social (TGSS) no dispone de una API pública web sencilla y directa, basada en estándares modernos como REST/JSON, que pueda ser consumida fácilmente desde plataformas en la nube [**blog_diario**] [3].

Tradicionalmente, esta comunicación requiere el uso de programas de escritorio específicos proporcionados por la administración, como la aplicación SILTRA o RED Directo [**blog_diario**] [2]. Estas herramientas exigen a los despachos laborales y gestorías manejar ficheros físicos, obtener autorizaciones previas como usuarios RED y, de forma imprescindible, gestionar certificados digitales locales para cada cliente de manera individual [**blog_diario**] [3]. Este enfoque representa un importante cuello de botella tecnológico, ya que dificulta la integración directa desde aplicaciones web o móviles modernas, exigiendo habitualmente el desarrollo de un *software* de comunicaciones especializado o la implantación de un *middleware* complejo [**blog_diario**]

[3].

A continuación, se muestra una imagen actual del proceso de comunicación con la TGSS a través del Sistema RED:

The screenshot shows the 'Oficina Virtual' interface for the Spanish Social Security system (TGSS). The header includes the Spanish flag, the text 'GOBIERNO DE ESPAÑA' and 'MINISTERIO DE EMPLEO Y SEGURIDAD SOCIAL', and the 'SEGURIDAD SOCIAL' logo. The main title is 'Oficina Virtual ALTAS Y BAJAS DE TRABAJADORES' with a 'Menú' button. Below the title, there are input fields for 'II. Afiliación', 'Régimen', and 'Cuenta de cotización'. The main section is titled 'ALTA' and contains a form with various fields for registration. The fields are organized into columns and rows, with some fields highlighted in red. At the bottom, there are buttons for 'Ayuda', 'Borrar', and 'Continuar', along with dropdown menus for 'Impresión Documento' and 'Impresión I.D.C.'.

Situación		Fecha Real Alta		Grupo Cotización	
01		13	03	2013	10
Ocupación		Tipo Contrato	421		Coef.T.Parcial 0,
Colectivo Trabajador		R.L.C.E			Cond. Desempleado 1
Mujer Subrepresentada	☐	Incap.Readmitido	☐		Excl.soc/Vio.dom
Trabajador de Autónomo	☐	Grado Minusvalía			Renta Activa Inserción
Causa Sustitución		NUSS Sustituido			Relevo
FIC Especifico	☐	Fec. Inicio Contrato			Parto Últimos 24 meses
Tipo Inactividad		Mujer Reincorporada			Vínculo Familiar
CESIÓN: Tipo		C.C.C			N.S.S
Cambio Puesto Trabajo		Convenio Colectivo			Pérdida Beneficios
Coef. Reductor Edad Jubil.		Cat. Profesional			Excedente Sec. Industr.
Fomento Empleo Agrario	☐				

Figura 2.1: Esquema del proceso de comunicación con la TGSS mediante el Sistema RED.

2.2. Soluciones alternativas y APIs de terceros

Dada la extrema dificultad técnica de integrar el Sistema RED directamente desde plataformas *frontend* o en la nube, el mercado ha evolucionado hacia la provisión de soluciones intermedias [blog_diario] [3, 4]. Existen empresas proveedoras de *software* de Recursos Humanos y nóminas que ofrecen servicios de pasarela y abstracción [blog_diario] [4]. Estas empresas exponen una API moderna (REST/JSON) orientada a desarrolladores, que se encarga de asimilar toda la complejidad del Sistema RED, manejando por detrás los certificados, los formatos de ficheros y los protocolos rígidos de la administración [blog_diario] [4, 5]. Un ejemplo destacado de este tipo de soluciones es la pasarela proporcionada por la empresa *CreateApps* [blog_diario] [10, 11].

El uso de estas APIs de terceros permite una integración sencilla con servicios en la nube (como Firebase Cloud Functions) mediante llamadas HTTP estándar [blog_diario] [5]. Sin embargo, presentan como desventaja una dependencia tecnológica vital hacia un proveedor de terceros para una funcionalidad crítica y un coste económico por transacción o suscripción [blog_diario] [5]. Debido a estas limitaciones y a los costes asociados, en el ámbito académico y de prototipado del presente trabajo, se ha optado

por el diseño y desarrollo de una *Mock API* (API simulada) desplegada de forma local [blog_diario, primera_entrega] [1, 11, 12]. Este componente emula las respuestas y validaciones de la administración pública para poder evaluar la robustez del sistema y el flujo funcional completo sin incurrir en dependencias ni gastos [blog_diario] [11, 12].

2.3. Tecnologías Low-Code y Backend-as-a-Service (BaaS)

En los últimos años, el desarrollo de aplicaciones empresariales ha adoptado enfoques orientados a reducir drásticamente el tiempo de comercialización (*Time to Market*) y la complejidad de la gestión de la infraestructura subyacente. El paradigma *Low-Code* permite la creación rápida de interfaces de usuario multiplataforma mediante un modelado visual [documento_diseno] [6]. En este contexto, *FlutterFlow* destaca como un entorno de desarrollo que genera código nativo estructurado basado en el *framework* Flutter, permitiendo construir el diseño UI/UX de formularios complejos y habilitar validaciones algorítmicas críticas en tiempo real de manera muy eficiente [documento_diseno, primera_entrega] [1, 6].

De forma complementaria, el estado del arte en la gestión de infraestructura en la nube se apoya en arquitecturas *Backend-as-a-Service* (BaaS) como Firebase, impulsado por Google [proyecto_alta] [7, 13]. Esta plataforma abstrae la administración de servidores dedicados y provee un ecosistema de servicios listos para su uso: Firebase Authentication para la gestión segura de accesos mediante roles; Cloud Firestore como base de datos NoSQL documental orientada a la escalabilidad en tiempo real; y Cloud Functions [documento_diseno, proyecto_alta] [6, 7]. Estas funciones en la nube (ejecutadas típicamente bajo Node.js) actúan como un *middleware* seguro, permitiendo centralizar la lógica de negocio compleja y las integraciones con APIs externas (como el SEPE o la TGSS) en un entorno protegido, evitando por completo la exposición de credenciales o secretos criptográficos en el lado del cliente [documento_diseno, proyecto_alta] [6, 7, 14].

2.4. Arquitecturas Multi-Tenant y Cumplimiento del RGPD

Para dar servicio a gestorías de Recursos Humanos que administran de forma simultánea a múltiples empresas cliente, los sistemas de información modernos deben implementar de base una arquitectura *Multi-Tenant* (multi-inquilino) [primera_entrega] [1]. En bases de datos de naturaleza NoSQL como Firestore, este modelo arquitectónico se resuelve utilizando patrones de referencias cruzadas entre documentos (*Document Reference*), asociando de manera inequívoca y relacional a cada empleado y cada usuario de la gestoría con la empresa o empresas a las que pertenecen [documento_diseno] [15, 16].

Este aislamiento y segregación estricta de la información es un requerimiento técnico innegociable para el cumplimiento íntegro del Reglamento General de Protección de Datos (RGPD) en el ámbito europeo [primera_entrega, proyecto_alta] [1, 9]. Al tratar diariamente con datos personales, fiscales y contractuales de alta sensibilidad (DNI/NIE, cuentas bancarias IBAN, afiliación a la Seguridad Social, contratos laborales), el sistema debe garantizar controles y mecanismos granulares de autorización [proyecto_alta] [8]. De este modo, un administrativo o usuario de Recursos

Humanos únicamente podrá generar registros de alta o visualizar el historial de las compañías a las que haya sido explícitamente autorizado por un administrador de sistema [**blog_diario**, **proyecto_alta**] [8, 17]. Adicionalmente, el RGPD y los marcos de buenas prácticas de auditoría exigen mantener una trazabilidad ininterrumpida de las operaciones sobre los datos, por lo que este tipo de arquitecturas incorporan de diseño colecciones históricas (tales como *altas_history*), encargadas de auditar de forma inmutable cada trámite, identificando el responsable de la acción, la empresa afectada, el estado del envío y la fecha de tramitación (sello temporal) [**blog_diario**, **proyecto_alta**] [18, 19].

Capítulo 3

Metodología y Herramientas

Este capítulo está dedicado a describir de forma detallada la metodología de trabajo seguida durante la realización de este Trabajo de Fin de Grado, así como a exponer exhaustivamente todas las herramientas informáticas empleadas. Estas herramientas abarcan desde el entorno de desarrollo y la infraestructura en la nube, hasta el software de gestión, colaboración y redacción del documento final.

3.1. Metodología de Desarrollo

Para la realización de este sistema se ha optado por un enfoque iterativo y un modelo de desarrollo ágil basado en la creación de prototipos funcionales (Prototipado). Este método permite disponer de vistas de interfaz navegables de forma temprana para validar los requisitos visuales antes de implementar la lógica de negocio subyacente.

La metodología AGILE es un marco de trabajo de ingeniería de software que promueve la planificación adaptativa, el desarrollo evolutivo y la mejora continua. Se basa en la entrega de valor de forma incremental y prioriza la respuesta rápida y flexible frente a los cambios imprevistos por encima de la rigidez de una planificación inicial cerrada. En el contexto de este proyecto, la filosofía ágil se ha utilizado para estructurar el trabajo en entregas funcionales a lo largo de las 14 semanas de planificación. En lugar de seguir un desarrollo lineal clásico o en cascada, se construyó de manera temprana el diseño visual en FlutterFlow para afianzar la experiencia de usuario con prototipos navegables, acoplando posteriormente y de forma progresiva la arquitectura NoSQL y la lógica en la nube mediante Firebase.

Esta gran capacidad de adaptación iterativa que ofrece AGILE resultó determinante para la consecución del proyecto; al descubrir en las fases de investigación que la Tesorería General de la Seguridad Social (TGSS) no ofrecía una API pública y directa para integrarse con su Sistema RED, el enfoque ágil permitió pivotar los objetivos rápidamente sin paralizar al resto de componentes. En consecuencia, el desarrollo de las integraciones se reorientó fluidamente hacia la creación de una Mock API simulada en el backend, manteniendo intacto el progreso paralelo de las tareas de frontend y del diseño de las bases de datos.

El ciclo de desarrollo se ha dividido en fases incrementales: comenzó con la revisión y diseño de la interfaz de usuario (UX/UI) en plataformas *Low-Code*, prosiguió con la implementación de funciones críticas y algoritmos de validación en el lado del cliente,

se complementó con la configuración de la arquitectura de bases de datos NoSQL y funciones en la nube, y finalizó con la integración y pruebas del sistema simulado. Esta metodología ha permitido adaptar el sistema de manera flexible a los estrictos requerimientos de segregación de datos que impone el RGPD.

3.2. Herramientas Empleadas

A continuación, se describen las diferentes herramientas empleadas en el proyecto, categorizadas según su propósito dentro del ciclo de vida del desarrollo y la gestión del trabajo.

3.2.1. Herramientas de Colaboración y Gestión Laboral

Para la planificación, comunicación y gestión diaria del proyecto, se ha hecho uso de las siguientes herramientas corporativas:

3.2.1.1. Microsoft Teams

Es una plataforma unificada de comunicación y colaboración empresarial de Microsoft que integra chats, videollamadas y almacenamiento de archivos en la nube. *Se ha elegido esta herramienta porque es el estándar corporativo y académico ideal para coordinar el trabajo, realizar reuniones de seguimiento con la dirección del proyecto y centralizar las dudas de manera fluida y asíncrona.*

3.2.1.2. Microsoft Word

Es un procesador de textos ampliamente utilizado que forma parte de la suite de Microsoft Office. *Se ha empleado principalmente en las fases iniciales del proyecto para la redacción de los documentos de análisis funcional, diseño de bases de datos y la propuesta inicial del TFG, dada su facilidad para el formateo rápido y la revisión de borradores.*

3.2.1.3. Microsoft PowerPoint

Es un software de presentación visual. *Se ha elegido para la creación de esquemas visuales, mapas conceptuales de alto nivel y, finalmente, para diseñar las diapositivas que apoyarán la defensa pública del Trabajo Fin de Grado ante el tribunal.*

3.2.1.4. Microsoft Excel

Es una aplicación de hojas de cálculo especializada en el manejo y análisis de datos estructurados. *Se ha utilizado en este proyecto para la revisión, limpieza y filtrado de los datos maestros descargados (como los listados de municipios, códigos postales y países), permitiendo eliminar duplicados y organizar la información antes de importarla a la base de datos.*

3.2.1.5. Microsoft Copilot

Es un asistente virtual impulsado por Inteligencia Artificial integrado en el ecosistema de Microsoft. *Se ha elegido como herramienta de apoyo para agilizar la resolución*

de dudas técnicas, la generación de ideas de estructuración de documentos y la depuración de pequeños fragmentos de código durante la fase de investigación.

3.2.2. Herramientas de Desarrollo Frontend

La interfaz con la que interactúan los usuarios de las gestorías se ha construido apoyándose en las siguientes tecnologías:

3.2.2.1. FlutterFlow

Es una plataforma de desarrollo visual *Low-Code* basada en el *framework* Flutter que permite construir interfaces de usuario multiplataforma (web y móvil) de manera gráfica. *Se ha elegido esta plataforma porque acelera drásticamente la creación del diseño UX/UI mediante componentes visuales, generando automáticamente código nativo y facilitando una integración directa, rápida y sin fisuras con el ecosistema de Firebase.*

3.2.2.2. Dart

Es un lenguaje de programación optimizado para el desarrollo de interfaces de usuario rápidas en múltiples plataformas, creado por Google y utilizado como motor detrás de Flutter. *Se ha elegido porque es el lenguaje nativo requerido por FlutterFlow para programar la lógica de programación y estructuras de datos en el lado del cliente (como las estrictas validaciones algorítmicas del formato DNI/NIE y el control de rangos de fechas).*

3.2.2.3. Material Design y Heurísticas de Nielsen

Material Design es un sistema de diseño de código abierto desarrollado por Google, orientado a guiar la creación de interfaces digitales consistentes, accesibles e intuitivas en múltiples plataformas. *Se ha seleccionado e integrado este marco de diseño debido a que FlutterFlow implementa sus componentes nativos bajo esta especificación de forma predeterminada.*

La adopción de Material Design no responde únicamente a criterios estéticos, sino que ha servido como base estructural para garantizar el cumplimiento de las 10 Heurísticas de Usabilidad de Jakob Nielsen durante la fase de prototipado del sistema. Al utilizar este ecosistema, la interfaz hereda de forma intrínseca patrones que satisfacen principios críticos evaluados en la disciplina de Interacción Persona-Ordenador, tales como:

- **Consistencia y estándares:** El uso de la paleta de componentes estandarizados de Material Design asegura que los elementos interactivos se comporten de manera predecible para el usuario a lo largo de toda la plataforma web y móvil.
- **Prevención y recuperación de errores:** Los campos de formulario y los diálogos de alerta integrados en el sistema de diseño facilitan la implementación de restricciones visuales inmediatas y mensajes claros de diagnóstico ante fallos.
- **Flexibilidad y eficiencia de uso:** La disposición adaptativa de los componentes permite optimizar los flujos operativos de las agencias de Recursos Humanos,

agilizando las interacciones recurrentes y la visualización de los datos maestros de cotización.

De este modo, el sistema de diseño actúa como un marco preventivo que minimiza los problemas de usabilidad antes de someter el prototipo de alta fidelidad a las fases de inspección formal y evaluación por pautas de accesibilidad.

3.2.3. Herramientas de Desarrollo Backend e Infraestructura

Para soportar la lógica de negocio segura, el almacenamiento y la arquitectura *Multi-Tenant*, se ha empleado la suite de Google:

3.2.3.1. Firebase

Es una plataforma en la nube orientada al desarrollo de aplicaciones (*Backend-as-a-Service*) que proporciona múltiples herramientas integradas de infraestructura. *Se ha elegido esta tecnología porque permite abstraer toda la gestión, mantenimiento y escalabilidad de los servidores físicos, ofreciendo un entorno seguro y unificado para la base de datos, autenticación y despliegue web.*

3.2.3.2. Firestore

Es una base de datos NoSQL documental, flexible y escalable alojada en la nube, perteneciente al ecosistema de Firebase. *Se ha elegido como núcleo del modelo de datos porque su estructura basada en colecciones, documentos y referencias cruzadas (Document Reference) se adapta de manera excelente a arquitecturas Multi-Tenant, garantizando la separación estricta de la información por empresa, requisito innegociable para cumplir con el RGPD.*

3.2.3.3. Firebase Authentication

Es el servicio de Firebase encargado de la identificación de usuarios y la gestión de contraseñas de forma segura. *Se ha elegido porque provee una solución robusta y lista para usar que se integra de manera nativa con las reglas de seguridad de Firestore, permitiendo establecer controles de acceso granulares basados en roles (administrador, RRHH, supervisor) de manera eficiente.*

3.2.4. Herramientas de Apoyo y Control de Versiones

3.2.4.1. GitHub

Es una plataforma de alojamiento en la nube orientada al desarrollo colaborativo de software que utiliza el sistema de control de versiones Git. *Se ha elegido esta herramienta porque representa el estándar de la industria para mantener un registro histórico inmutable de los cambios en el código, garantizando la seguridad del código fuente generado y permitiendo recuperar versiones estables del proyecto en caso de errores críticos.*

También ha sido usada para almacenar de forma segura las copias de seguridad de la redacción del TFG, asegurando que no se pierda ningún avance significativo en la elaboración de la memoria académica.

3.2.4.2. Script de Automatización

Enlazando con GitHub, se ha desarrollado un script de automatización que, cada vez que se ejecuta, hace una copia de seguridad del entorno de trabajo y desarrollo de este documento, subiéndolo automáticamente a un repositorio privado en GitHub. *Se ha implementado esta herramienta para garantizar que cada avance significativo en la redacción del TFG quede registrado de forma segura, permitiendo recuperar versiones anteriores en caso de errores o pérdidas accidentales de datos.*

3.2.5. Herramientas de Redacción y Formato

Para la elaboración final de la memoria académica se han utilizado los siguientes recursos:

3.2.5.1. Visual Studio Code

Es un editor de código fuente ligero, extensible y altamente personalizable. *Se ha elegido porque ofrece un soporte excelente mediante extensiones creadas por la comunidad, resultando ser la herramienta idónea tanto para examinar y procesar los grandes archivos JSON de la base de datos, como para escribir y compilar el documento final de la memoria.*

3.2.5.2. LaTeX

Es un sistema de composición de textos de alta calidad, diseñado para la producción de documentación técnica y científica. *Se ha elegido en las fases finales del proyecto porque permite separar el contenido del formato visual, garantizando una presentación tipográfica profesional, una gestión impecable de la bibliografía y un cumplimiento riguroso de los estándares de estilo académicos exigidos por la universidad.*

Capítulo 4

Módulo de Gestión

Este primer módulo, denominado módulo de gestión, constituye el núcleo de la plataforma. Está dedicado íntegramente a la gestión de la aplicación, incluyendo la administración de usuarios, la autenticación y la arquitectura visual y de datos asociada al inicio del sistema.

4.1. Gestión de Usuarios y Autenticación

4.1.1. Proceso de Login y Peticiones a Firebase

Este módulo se encargará del login de los usuarios autorizados en mi empresa. Para agilizar y facilitar el proceso, el usuario deberá introducir tan solo su correo y contraseña para acceder a su homepage.

4.1.1.1. Uso de Firebase Authentication

Usamos Firebase Authentication para gestionar el sistema de accesos de manera integral y segura. Cuando un usuario introduce su correo y contraseña, la aplicación cliente envía una petición de autenticación (*signInWithEmailAndPassword*) a los servidores de Firebase. Si las credenciales son válidas, el servicio responde con un token cifrado de sesión y un identificador único (UID). Por otro lado, para registrar nuevos perfiles en la plataforma, el sistema lleva a cabo peticiones de creación de cuentas (*createUserWithEmailAndPassword*), enlazando posteriormente esa identidad con el documento de permisos en la base de datos. También se gestionan a través de este servicio las peticiones para el restablecimiento de contraseñas olvidadas.

4.1.1.2. Tipos de Usuarios

De entre los tres tipos de usuarios que se validan a través de este módulo, distinguimos los siguientes niveles de acceso y gestión:

- **Administrador:** El “admin” es el rol con mayores privilegios de la plataforma; podrá dar de alta tanto a empresas, como a usuarios, asignándoles el acceso correspondiente.

- **Recursos humanos:** El usuario “rrhh” tiene un perfil totalmente operativo; debería solo poder gestionar los usuarios de las empresas autorizadas a este, garantizando la privacidad de los datos.
- **Supervisor:** Considero que el supervisor debe ser un usuario que, dentro de la empresa, pueda dar de alta a más usuarios de rrhh. Esto permite delegar la administración del equipo interno a los responsables de las gestorías sin depender de un administrador global del sistema.

4.1.2. Delegación de Permisos

La jerarquía descrita mediante los distintos roles permite una delegación de permisos segura y estructurada. Una vez autenticado, el sistema reconoce inmediatamente el nivel de acceso del individuo y bloquea o permite las funcionalidades correspondientes para mantener un entorno de trabajo unificado pero completamente compartido.

4.2. Estructura del Frontend

El frontend de este módulo, implementado con tecnología *Low-Code*, está estructurado principalmente en flujos de navegación que diferencian las áreas públicas de las privadas. La estructura del frontend se inicia en la **Login Page**, una vista sencilla que aloja el formulario de credenciales (correo y contraseña). No hay validaciones en tiempo real para el formato del correo o la fortaleza de la contraseña, ya que estas validaciones se delegan completamente a Firebase Authentication, que se encarga de verificar la existencia del usuario y la validez de las credenciales.

Una vez superado el login, el usuario es redirigido a la **Homepage**. La estructura interna de esta pantalla privada incluye un menú de navegación lateral (Drawer) fijo y un cuerpo central. La interfaz hace un fuerte uso de lógicas de **visibilidad condicional**; dependiendo del tipo de usuario (Admin, RRHH, Supervisor) que haya iniciado sesión, la estructura del frontend oculta o muestra automáticamente las distintas pantallas de gestión (como el formulario para dar de alta a una nueva empresa, que solo será visible para el administrador).

4.3. Estructura del Backend

El backend para este módulo funciona conectando Firebase Authentication con la base de datos documental (Firestore). Una vez se completa el registro o login de un usuario, el backend lleva a cabo la recuperación de un documento maestro en la colección *users* que contiene la información completa del perfil, el rol asignado y la referencia a su empresa autorizada. Para soportar las lógicas de segregación y seguridad (Multi-Tenant), en este apartado del backend cobra especial importancia el uso de **índices**. Dado que un usuario de rrhh solo debe acceder a los registros vinculados a la empresa que tiene autorizada, el sistema lleva a cabo consultas con filtros combinados (ej. buscar usuarios donde el campo *empresa_ref* sea X, y el campo *rol* sea Y). Firestore exige la creación de **índices compuestos** específicos en el backend para poder estructurar previamente estos datos en memoria. La implementación de estos índices permite que las búsquedas y validaciones de permisos se ejecuten de

Módulo de Gestión

manera casi instantánea y altamente eficiente, asegurando que nadie pueda realizar cruces de información no permitidos entre distintas empresas.

Capítulo 5

Módulo de Autenticación y Control de Roles

Este capítulo está dedicado a describir y explicar el desarrollo de la parte de autenticación del sistema, detallando el control de roles de usuario, la gestión de permisos, el diseño de la pantalla principal y cómo se estructuran los datos de registro dividiéndolos entre el servicio de autenticación y la base de datos documental.

5.1. Pantalla Principal y Opciones de Usuario

Una vez que el usuario inicia sesión de manera exitosa, entrará directamente en su página de inicio o “homePage”. En este panel se mostrarán los datos principales del perfil autenticado, tales como el nombre y los permisos que posee. Además, desde aquí se tendrá acceso a diferentes botones y funcionalidades, cuya visibilidad estará condicionada en función de los permisos del usuario.

A continuación, se muestra una imagen de la pantalla de inicio de sesión de la aplicación, donde se pueden observar los campos para introducir el correo electrónico y la contraseña, así como el botón de inicio de sesión:

Alta Seguridad Social

Inicio de sesión

Inicie sesión con su correo y contraseña.

Correo

Contraseña

Iniciar sesión

Me olvidé de la contraseña

Or sign up with

Continue with Google

Continue with Apple



Figura 5.1: Pantalla de inicio de sesión

El diseño de la interfaz de acceso se ha concebido bajo un enfoque de minimalismo funcional y estética corporativa. Se ha optado por una distribución asimétrica en dos bloques diferenciados: el panel izquierdo concentra exclusivamente la carga interactiva, mientras que el panel derecho actúa como un elemento visual de soporte que refuerza la identidad profesional de la plataforma. Con esta estructura se consigue maximizar el espacio en blanco y reducir el ruido visual, guiando de forma directa la atención del usuario hacia los campos de entrada de datos. Este diseño prioriza una experiencia de usuario ágil y directa, evitando elementos distractores en una sección tan crítica como el control de acceso. Como bien se ha mencionado en el apartado 4.1, no se hacen validaciones de formato en tiempo real porque dichas validaciones se harán a través del formulario de creación de usuarios.

5.1.1. Opciones de Administrador

Los usuarios que cuenten con el rol de administrador dispondrán de capacidades de gestión de entidades y personal, teniendo acceso a las siguientes opciones:

- **Dar de alta a empresas:** El administrador, mediante un formulario para recoger todos los datos necesarios de la entidad, podrá dar de alta a nuevas empresas en la aplicación para que estas puedan usar el servicio.
- **Dar de alta a usuarios:** El administrador también podrá registrar a un nuevo usuario para que este pueda, de forma exclusiva para la empresa o empresas seleccionadas, dar de alta a nuevos empleados.

5.1.2. Opciones de Supervisor

Los usuarios con el rol de supervisor tendrán acceso a las mismas opciones que los administradores, pero con una limitación importante: solo podrán dar de alta a nuevos usuarios de recursos humanos (RRHH) dentro de su ámbito de acceso autorizado. Es decir, un supervisor no podrá registrar a nuevos administradores ni modificar los permisos de acceso a empresas, pero sí podrá gestionar la creación de perfiles ope-

rativos de RRHH para facilitar la administración interna de la empresa o empresas a las que tiene acceso autorizado. Este es el rol que tendrán dentro de la aplicación los gestores de las gestorías, ya que podrán gestionar a su vez a los usuarios de RRHH de sus clientes, pero no tendrán acceso a la gestión de empresas ni a la creación de administradores globales del sistema.

Esta delegación de permisos permite una gestión más eficiente y segura del sistema, evitando que se otorguen privilegios excesivos a usuarios que no necesitan acceso completo a la plataforma.

5.1.3. Opciones del Usuario de Recursos Humanos

Por otro lado, los usuarios de recursos humanos (RRHH) tendrán acceso a las operativas de registro de trabajadores, divididas en las siguientes acciones:

- **Seleccionar empresa:** De entre todas las empresas a las que el usuario de RRHH tiene acceso autorizado, este deberá escoger aquella en la que quiera tramitar un alta.
- **Dar de alta:** Una vez escogida la empresa, se procederá a recoger los datos personales y contractuales necesarios del empleado para formalizar su registro.

5.2. Gestión de Datos y Formulario Dual

Para dar de alta a un usuario (o empleado), el administrador debe rellenar un formulario específico. Este formulario va a tener una doble función arquitectónica, debido a que el servicio de Google Auth no permite añadir campos personalizados adicionales al perfil básico. Por un lado, se guardarán los datos de acceso en Google Auth (estos son el correo y el hash de la contraseña), garantizando la encriptación de la contraseña; por otro lado, el resto de los campos operativos se guardarán dentro de la base de datos Firestore bajo la colección *users*.

5.2.1. Almacenamiento en Google Auth

A la plataforma de autenticación de Google se enviarán exclusivamente los datos críticos de acceso:

- **Correo:** El correo electrónico del usuario, que se utilizará como identificador para el inicio de sesión.
- **Hash de contraseña:** La contraseña del usuario, que se almacenará de forma segura y cifrada por Google Auth, garantizando la protección de esta información sensible.

Es importante recalcar que, aunque el correo se almacena en Google Auth, este mismo dato también se guarda en la base de datos documental (Firestore) para poder realizar las validaciones de permisos y roles de manera eficiente.

Sin embargo, la contraseña no se almacena en Google Auth, sino que se guarda un “resumen” o “hash” de la misma con técnicas de cifrado avanzadas (bcrypt), lo que garantiza que incluso en caso de una brecha de seguridad, las contraseñas de los usuarios no puedan ser recuperadas ni utilizadas por terceros malintencionados.

Módulo de Autenticación y Control de Roles

Además, dos contraseñas idénticas de diferentes usuarios no tendrán el mismo hash, porque se utiliza un proceso de salting que añade un valor aleatorio a cada contraseña antes de cifrarla, lo que aumenta significativamente la seguridad del sistema de autenticación.

En su lugar, Google Auth devuelve un token de sesión cifrado y un identificador único (UID) que se utilizan para vincular el perfil de autenticación con el documento de permisos en Firestore, garantizando así una gestión segura y eficiente de los datos de acceso. Esto se usará para validar el acceso a la aplicación y para gestionar los permisos de cada usuario, asegurando que solo aquellos con las credenciales correctas puedan acceder a las funcionalidades correspondientes a su rol.

A continuación, se muestran dos imágenes (una de Google Auth y otra de Firestore), una por encima de la otra, para ilustrar cómo se dividen los datos entre ambos servicios y cómo se complementan para ofrecer una solución de autenticación robusta y segura, omitiendo por motivos de seguridad algunos campos:



Identificador	Proveedores	Fecha de creación ↓	Fecha de acceso	UID del usuario
[REDACTED]	[REDACTED]	[REDACTED]	[REDACTED]	[REDACTED]
[REDACTED]	[REDACTED]	[REDACTED]	[REDACTED]	[REDACTED]
[REDACTED]	[REDACTED]	[REDACTED]	[REDACTED]	[REDACTED]
[REDACTED]	[REDACTED]	[REDACTED]	[REDACTED]	[REDACTED]
[REDACTED]	[REDACTED]	[REDACTED]	[REDACTED]	[REDACTED]
[REDACTED]	[REDACTED]	13 oct 2025	4 may 2026	TUFCFqACEZU07bfZUtriJT22...

Buscar por dirección de correo electrónico, número de teléfono o UID de usuario

Agregar usuario

Filas por página: 50 1 - 6 of 6

Figura 5.2: Datos almacenados en Google Auth

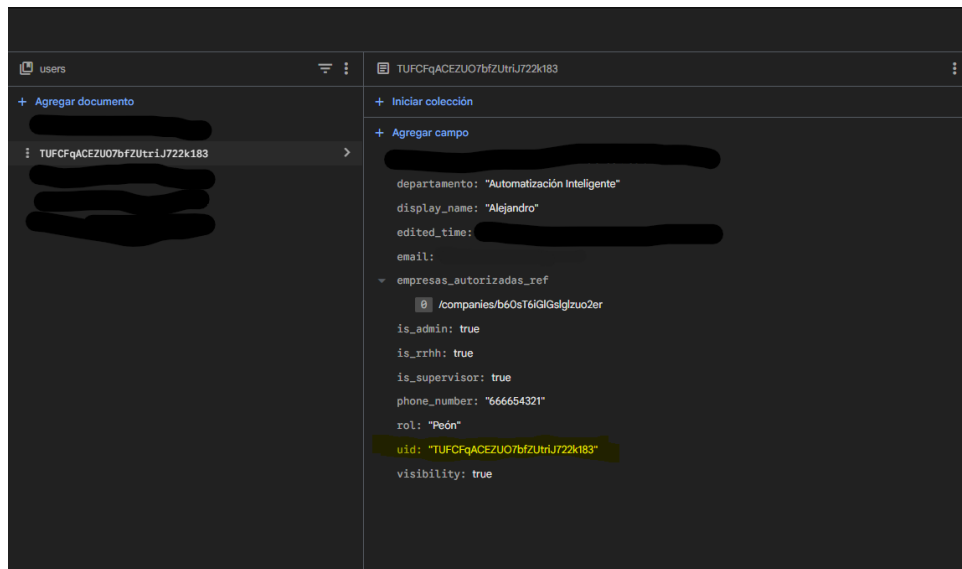


Figura 5.3: Datos almacenados en Firestore

5.2.2. Almacenamiento en Firestore

Toda la información adicional y el control de roles se gestiona como documentos en la base de datos NoSQL Firestore.

Se almacena un documento por cada usuario registrado, el cual contiene campos específicos para identificar su rol (administrador, RRHH, supervisor) y las empresas a las que tiene acceso autorizado. Este enfoque permite una gestión flexible y escalable de los permisos, ya que se pueden añadir o modificar campos sin afectar la estructura general de la base de datos. Además, se implementan índices compuestos para optimizar las consultas y garantizar que los usuarios solo puedan acceder a la información de las empresas que tienen autorizadas, manteniendo así la seguridad y privacidad de los datos. Esto último es especialmente importante para cumplir con el RGPD, ya que garantiza que los datos de cada empresa estén segregados y solo accesibles para los usuarios autorizados, evitando cualquier riesgo de acceso no autorizado o violación de la privacidad.

A continuación, se muestra una imagen del formulario de registro de un nuevo usuario, donde se pueden observar los campos que se rellenan para almacenar la información tanto en Google Auth como en Firestore:

Figura 5.4: Formulario de registro de un nuevo usuario

Como se puede observar en la imagen, el formulario recoge tanto los datos de acceso (correo y contraseña) que se envían a Google Auth, como los datos adicionales (rol y empresas autorizadas) que se almacenan en Firestore. Esta estructura dual permite una gestión eficiente y segura de la autenticación y los permisos de los usuarios dentro de la plataforma.

Además, se puede observar que la casilla de “Recursos humanos” está marcada por defecto, al contrario que las de “Administrador” y “Supervisor”, que están desmarcadas. Esto se debe a que, en la mayoría de los casos, el perfil de RRHH es el más común entre los usuarios de la plataforma, mientras que los roles de administrador y supervisor suelen ser más limitados y específicos. A pesar de ello, por ser una casilla de tipo checkbox, el administrador tiene la libertad de marcar o desmarcar cualquiera de las opciones según las necesidades de cada usuario registrado.

5.2.2.1. Campos de Identificación y Roles

Dentro del documento de cada usuario en Firestore se almacenan los siguientes campos:

- **Correo:** El mismo correo que se usará para el inicio de sesión, actuando como identificador común.

- **Is_admin:** Un campo lógico (cierto o falso). Se utilizará para autorizar la vista u oculte de botones a los que un usuario de la aplicación puede acceder.
- **Is_rrhh:** Un campo de valor lógico similar al anterior, que identifica a los usuarios operativos de recursos humanos.
- **Is_supervisor:** El tercer campo de valor cierto o falso. Aquellos usuarios con este campo con valor cierto tendrán privilegios para registrar a otros usuarios de RRHH dentro de su ámbito.

5.2.2.2. Control de Acceso Multi-Tenant

Para garantizar la segregación de la información de las distintas organizaciones, se utiliza un campo adicional:

- **Empresas autorizadas:** Un campo estructural que se utilizará para gestionar e identificar a qué empresas específicas tiene acceso un usuario.

5.2.2.3. Lista de datos de un usuario

A continuación, se muestra una lista con todos los campos que se almacenan en el documento de cada usuario en Firestore, incluyendo el tipo de dato y su función dentro del sistema de autenticación y control de roles:

- **Correo (String):** El correo electrónico del usuario, que también se almacena en Google Auth para la autenticación.
- **UID (String)** El identificador único que devuelve Google Auth, utilizado para vincular el perfil de autenticación con el documento de permisos en Firestore.
- **Created_time (Timestamp):** La fecha y hora en que se creó el usuario, útil para auditorías y gestión de usuarios.
- **Edited_time (Timestamp):** La fecha y hora de la última modificación del perfil del usuario, para mantener un registro actualizado. Este campo no aparecerá en el fichero hasta que no se modifique el perfil por primera vez.
- **Phone_number (String):** Un campo opcional para almacenar el número de teléfono del usuario, que puede ser utilizado para comunicaciones internas o autenticación de dos factores.
- **is_admin (Boolean):** Explicado anteriormente.
- **is_rrhh (Boolean):** Idem al anterior, pero para identificar a los usuarios de recursos humanos.
- **is_supervisor (Boolean):** Idem al anterior, pero para identificar a los usuarios con privilegios de supervisión.
- **Empresas autorizadas_ref (Vector de referencias de empresas):** Un campo estructural (array) que contiene referencias a los documentos de las empresas a las que el usuario tiene acceso autorizado, utilizado para gestionar el control de acceso multi-tenant.
- **Rol (String):** Un campo en el que se explica cuál es la función del usuario dentro de la empresa, como por ejemplo “Jefe de RRHH”, “Supervisor de nóminas”,

etc. Este campo es meramente informativo y no se utiliza para la gestión de permisos, pero puede ser útil para identificar el perfil profesional del usuario dentro de la organización.

- **Departamento (String):** Similar al campo de rol, este campo describe a qué departamento pertenece el usuario dentro de la empresa, como por ejemplo “Recursos Humanos”, “Finanzas”, etc. Al igual que el campo de rol, es un dato informativo que no afecta a la gestión de permisos, pero puede ser útil para segmentar y organizar a los usuarios dentro de la plataforma.
- **Visibility (Boolean):** Un campo lógico que se utiliza para controlar la visibilidad del perfil del usuario dentro de la plataforma. Si este campo tiene un valor de falso, el perfil del usuario no será visible para otros usuarios, lo que puede ser útil para gestionar usuarios inactivos o para mantener la privacidad de ciertos perfiles dentro de la organización. Esto se usa para “borrar” usuarios sin eliminar completamente su información de la base de datos, garantizando la trazabilidad y la integridad de los datos, cumpliendo así con las normativas de protección de datos y facilitando la gestión de usuarios a largo plazo.

5.3. Cumplimiento del RGPD y Asignación Manual

El flujo de asignación de empresas a los usuarios conlleva un protocolo estricto. Inicialmente, se dará de alta a un usuario manualmente, dejando vacío el campo de empresas autorizadas. Una vez que el usuario ha sido registrado exitosamente, es el superusuario encargado del control y mantenimiento de este servicio quien debe añadir manualmente el identificador (ID) de la empresa o empresas a las que este usuario tiene acceso.

Este procedimiento debe realizarse obligatoriamente de esta manera para garantizar que no se viole el Reglamento General de Protección de Datos (RGPD). De no seguirse este control estricto, cualquier administrador podría dar acceso a un usuario a los datos confidenciales de otra compañía (como, por ejemplo, la empresa de la competencia). En pocas palabras, el dato “Empresas autorizadas” es un dato exclusivo de backend, protegido contra asignaciones automáticas no verificadas.

Una vez que el superusuario ha asignado manualmente el acceso a las empresas correspondientes, se consideran dos escenarios posibles para los usuarios de RRHH, dependiendo de si tienen o no el campo ‘Is_supervisor’ con valor cierto o falso:

- **Is_supervisor = true:** Si el usuario tiene el campo ‘Is_supervisor’ con valor cierto, tendrá privilegios para registrar a otros usuarios de RRHH dentro de su ámbito. Es decir, podrá dar de alta a nuevos usuarios de RRHH y asignarles el mismo acceso a empresas que él tiene autorizado, pero no podrá modificar los permisos de acceso a empresas ni asignar permisos de administrador.
- **Is_supervisor = false:** Si el usuario no tiene el campo ‘Is_supervisor’ con valor cierto, entonces este usuario no tendrá privilegios para registrar a otros usuarios de RRHH dentro de su ámbito. Solo podrá acceder a las funcionalidades operativas de recursos humanos, como dar de alta a empleados dentro de las empresas a las que tiene acceso autorizado, pero no podrá gestionar ni asignar permisos a otros usuarios.

Capítulo 6

Módulo de Gestión de Empresas

Este capítulo está dedicado a detallar el tercer módulo principal del sistema, centrado en la gestión integral de las distintas empresas clientes. En primer lugar, se abordan los procesos de creación y gestión de dichas entidades según los permisos asignados al usuario. A continuación, se detalla exhaustivamente la estructura y categorización de los datos maestros necesarios que conforman el formulario de registro de la empresa para cumplir con los requerimientos técnicos y legales.

6.1. Operativas de Gestión

El sistema diferencia la interacción con las empresas clientes de acuerdo con el rol y el nivel de acceso concedido al usuario en el módulo de gestión de identidades.

6.1.1. Creación de Nueva Empresa

La capacidad de dar de alta a nuevas empresas está restringida exclusivamente a los usuarios con permisos de “admin” (administrador). Esta acción se materializa mediante la cumplimentación de un formulario detallado de empresa en el frontend. En dicho proceso, que corresponde a los requerimientos abordados en las primeras etapas del desarrollo, el administrador debe introducir de manera precisa toda la información legal, fiscal y de cotización necesaria para habilitar la operativa de la empresa dentro de la plataforma.

A continuación, se adjunta una captura de pantalla del formulario de alta de empresa, que ilustra la interfaz y los campos requeridos para completar el proceso de registro:

Antites

Navegación de la plataforma

- Usuarios
- Empresas
- Empleados

Configuración

- Ayuda
- Configuración
- [Cerrar sesión](#)

Light Mode Dark Mode

Andrew D.
admin@gmail.com

Crear empresa ①

Razón social

CIF

Domicilio Social

ID Municipio

ID Municipio Cotización

Regimen

Código Cuenta Cotización

Código nacional de Actividades Económicas (CNAE)

Actividad Económica

Clave percepto

Administración de Hacienda

+ Añadir empresa < Volver atrás

Figura 6.1: Formulario de Alta de Empresa

6.1.2. Edición y Gestión de Empresas Existentes

Además de la creación, los usuarios con permisos de “admin” también pueden acceder a la gestión de las empresas ya registradas. Esto incluye la capacidad de editar los datos maestros de la empresa, actualizar información relevante y gestionar aspectos relacionados con la operativa diaria. La edición se realiza a través de una interfaz similar al formulario de alta, pero con los campos pre-cumplimentados con los datos existentes, permitiendo así una actualización eficiente y sin errores.

Adjuntamos a continuación una captura de pantalla de la interfaz de edición de empresa, que muestra cómo se presentan los datos para su modificación:

Figura 6.2: Interfaz de Edición de Empresa

6.2. Datos del Formulario de Empresa

El formulario de alta de empresa cumplimentado por el administrador recoge una extensa batería de datos maestros, indispensables para que el sistema y sus futuras integraciones operen sin errores. Estos datos se dividen lógicamente en diversas secciones principales:

6.2.1. Identificación y Datos de Registro

Esta categoría engloba la información legal primaria que define y ubica a la entidad corporativa.

6.2.1.1. Identificación Corporativa

El formulario exige la generación de un **ID de la Empresa**, el cual actúa como clave única dentro del sistema, siendo el pilar fundamental para garantizar la segregación de datos (arquitectura Multi-Tenant). A nivel formal, se recoge la **Razón Social**, correspondiente al nombre legal completo usado obligatoriamente en contratos, nóminas y trámites del SEPE; así como el **CIF** (Código de Identificación Fiscal), necesario para la identificación ante la Agencia Tributaria.

6.2.1.2. Información de Domicilio

Se solicita el **Domicilio Social**, que establece la dirección legal de la sede para el envío de notificaciones. En vinculación a la dirección, se debe introducir el **ID del Municipio** (del domicilio social), que es clave para ciertos trámites y para la identificación geográfica requerida por el SEPE. Paralelamente, se debe facilitar el **ID del Municipio de cotización**, el cual puede ser distinto al domicilio social y es de uso

estricto por parte de la Tesorería General de la Seguridad Social (TGSS) para los trámites específicos de afiliación, pero esta asignación se realiza de manera automática en el backend, vinculando el municipio de cotización al municipio del domicilio social a través de una tabla de correspondencia predefinida.

6.2.2. Parámetros de Cotización y Actividad (TGSS)

Este bloque abarca todos aquellos parámetros necesarios para interactuar con los organismos oficiales correspondientes, tanto en materia de Seguridad Social como fiscal.

6.2.2.1. Datos de Afiliación a la Seguridad Social

El sistema requiere establecer el **Régimen** de la Seguridad Social aplicable a la empresa (como el General o el Especial Agrario), siendo normalmente el "general" para la gran mayoría de organizaciones. Indisolublemente ligado a esto se encuentra el **Código de Cuenta de Cotización (CCC)**, una clave alfanumérica única que identifica a la empresa ante la Seguridad Social, asociando el centro de trabajo y la mutua, y constituyendo la llave indispensable para procesar cualquier alta.

6.2.2.2. Actividad y Fiscalidad

Para clasificar la labor de la entidad, se recogen tanto el **Código de actividad económica (CNAE)** como la **Actividad económica** (la descripción literal basada en el CNAE), siendo ambos campos obligatorios en el registro. A nivel tributario, el administrador debe aportar la **Clave percepto**, utilizada por Hacienda para identificar la naturaleza de ingresos o pagos y aplicar retenciones; y la identificación de la **Administración de Hacienda** a la que está adscrita la sede para la gestión documental. Finalmente, se recoge el **Código Idioma MEC** para regular el idioma de comunicaciones con el Ministerio de Empleo y Seguridad Social.

6.2.3. Datos de Contratos y Convenios

Esta sección asienta las bases legales bajo las cuales la empresa conformará las relaciones laborales con sus futuros empleados.

6.2.3.1. Convenio Aplicable

Se debe especificar el **Código convenio**, un código unívoco que indica qué convenio colectivo aplica a la empresa, el cual resulta imprescindible para clasificar correctamente al trabajador en el sistema de la TGSS. Este campo se complementa con la entrada de texto del **Convenio colectivo** (su denominación oficial), una información de obligada presencia en la elaboración física o digital del contrato de trabajo.

6.2.3.2. Estructura Salarial

El formulario documenta los **Conceptos salariales**. Esta sección define la estructura base de las futuras nóminas de la empresa, delimitando componentes como el sueldo base, complementos salariales y pagas extraordinarias. Estos datos serán el punto de partida fundamental para calcular las bases de cotización y para interconectar el proceso de alta con un futuro módulo de gestión de nóminas.

6.2.4. Datos de Firma y Documentación

Esta última categoría abarca los datos necesarios para formalizar la representación legal de la empresa, la automatización de la documentación interna y el estricto cumplimiento normativo.

6.2.4.1. Firma y Representación Legal

Para la rúbrica de contratos y comunicaciones, el sistema requiere el **NIF Firmante**, correspondiente a la persona legalmente autorizada para firmar los contratos. Este dato es un requisito ineludible para la correcta tramitación y resguardo de la comunicación del contrato al SEPE mediante el sistema Contrat@. A este identificador se le vincula el **Concepto Firmante**, que establece el cargo de dicho individuo (por ejemplo, “Apoderado” o “Gerente”) para su figuración en el contrato. Adicionalmente, se solicita la **Firma Copia básica**, que guarda la ruta o referencia a la imagen de la firma del responsable. Este recurso se utilizará en caso de que la aplicación automatice la impresión o generación del contrato destinado a la Representación Legal de los Trabajadores (RLT).

6.2.4.2. Gestión Documental y Privacidad

Para agilizar el flujo de trabajo de la gestoría, se parametriza un **Formato Carta**, consistente en una plantilla predefinida que la plataforma usará de manera interna para generar notificaciones o documentación rápida. Del mismo modo, se establece una **Ruta archivo**, que define el directorio del sistema donde se alojarán los archivos maestros de la empresa (como los modelos de contrato base), asegurando así una gestión documental organizada y con plena trazabilidad. Finalmente, y para garantizar la adecuación a la normativa de privacidad, se debe introducir el **ID de cláusula**. Este identificador relaciona a la empresa con sus cláusulas de Protección de Datos (RGPD) específicas, permitiendo que el formulario de alta incluya de forma automática los textos legales requeridos (según el art. 13 del RGPD) al incorporar a cualquier nuevo trabajador.

Capítulo 7

Módulo de Gestión de Empleados

Este capítulo está dedicado a detallar el cuarto módulo principal del sistema, el cual se centra en la gestión de los empleados asociados a una empresa. En primer lugar, se explica la operativa y los permisos necesarios para llevar a cabo el registro de un nuevo trabajador, así como la posterior edición de su información. A continuación, se detalla de forma exhaustiva la estructura y categorización de los datos que conforman el formulario de alta de empleados, abarcando desde la identificación básica hasta los datos de formación, detalles de contrato y las constantes internas necesarias para las integraciones oficiales.

7.1. Operativa de Empleados: Alta y Edición

La funcionalidad troncal de este módulo es la creación de nuevos registros de trabajadores para una entidad u organización específica, así como el mantenimiento seguro de dicha información a lo largo del tiempo.

7.1.1. Proceso de Registro y Permisos (revisar)

Para poder crear un empleado, el usuario debe contar con permisos de “rrhh” (recursos humanos). Una vez que el usuario cuenta con este rol, está autorizado para acceder al registro de nuevos empleados a través de un formulario específico en el frontend. Este formulario recoge toda la información legal, de contacto, residencia y contractual del trabajador, que posteriormente se comunicará a la TGSS para formalizar el alta técnica.

7.2. Datos del Formulario de Empleados

The screenshot displays the 'Antites' employee registration interface. On the left, a sidebar menu includes 'Usuarios', 'Empresas', and 'Empleados' under 'Navegación de la plataforma', and 'Ayuda' and 'Configuración' under 'Configuración'. A 'Cerrar sesión' button is also present. The main content area is titled 'Antites' and contains three sections: 'Datos de contacto' with fields for DNI/NIE, NAF, Nombre, Primer apellido, Segundo apellido, Email, Sexo, Teléfono, Fecha de nacimiento, and Sigla país de nacimiento; 'Datos de domicilio' with fields for Numero, Piso, Puerta, Código postal, Nombre del municipio, and Identificador de municipio; and 'Datos de contrato y formación' with fields for Fecha de alta and Fecha de baja. The user profile at the bottom left shows 'Andrew D. admin@gmail.com'.

Figura 7.1: Formulario de Alta de Empleado

Tras completar el formulario y confirmar el registro, el sistema enviará al endpoint facilitado por la empresa la información del nuevo empleado, lo que a su vez desencadenará la comunicación automática con la TGSS para formalizar el alta técnica. Es importante destacar que el proceso de alta en la plataforma y el proceso de alta en la Seguridad Social son dos acciones distintas pero interconectadas, siendo esta última una consecuencia directa del registro en el sistema.

7.1.2. Edición de Datos y Trazabilidad

Si se ha cometido un error en la introducción de datos o si es necesario actualizar la información de un empleado, el usuario con permisos de "rrhh" también tiene la capacidad de editar los registros existentes. La edición se realiza a través de una interfaz similar al formulario de alta, pero con los campos pre-cumplimentados con los datos actuales del empleado, lo que permite una actualización eficiente y sin errores.

Al igual que en el capítulo anterior, el menú de edición de empleados difiere del de creación en que aparece un botón en la zona superior derecha para eliminar al empleado, y en que, en vez de aparecer en la parte inferior de la página el botón de "Añadir usuario", aparece un botón de "Guardar cambios" para confirmar la actualización de los datos.

7.2. Datos del Formulario de Empleados

En esta sección, se va a explicar en detalle qué datos se recogen en el formulario. Aunque ya se ha hablado de ello en los apartados anteriores, las imágenes adjuntadas en dichos apartados no muestran al completo el formulario, por lo que se ha considerado necesario dedicar un apartado específico para detallar cada uno de los campos que se recogen en el mismo.

Para formalizar el registro o la edición, el usuario debe rellenar un formulario que recoge toda la información legal, de contacto, residencia y contractual del empleado.

Estos datos se estructuran lógicamente en los siguientes apartados:

7.2.1. Datos de Identificación y Contacto

Esta sección agrupa la información personal primaria del trabajador, así como los identificadores necesarios para los trámites legales y de afiliación a los organismos públicos.

7.2.1.1. Identificación Oficial y Afiliación

El sistema requiere especificar en primer lugar el **Tipo de Documento** de identificación (por ejemplo, DNI, NIE o Pasaporte). Esta selección define qué formato y valor se guardará en el campo contiguo, el **DNI/NIE**, el cual actúa como la clave principal de identificación fiscal y laboral del empleado. Adicionalmente, es obligatorio introducir el **NAF** (Número de Afiliación a la Seguridad Social), un identificador único del trabajador ante la Tesorería General de la Seguridad Social (TGSS), indispensable para tramitar y validar cualquier alta.

7.2.1.2. Filiación y Datos Personales

Se deben recoger los datos básicos de filiación: **Nombre, Apellido1 y Apellido2**, los cuales sirven como identificación formal en el contrato, en la propia comunicación del alta y en la posterior generación de nóminas. También se solicita el **Sexo** (género del trabajador), un dato necesario para cumplimentar el Modelo 145 del IRPF y para ciertos trámites estadísticos de la TGSS. Otro campo estrictamente obligatorio es la **Fecha de nacimiento**, fundamental para la verificación de la identidad y de la edad legal del trabajador ante la Seguridad Social. Por último, se debe indicar el **ID País**, que identifica el país de nacionalidad del empleado y es crucial para diferenciar entre trabajadores comunitarios y extracomunitarios.

7.2.1.3. Información de Contacto

El formulario incluye un campo para el **Teléfono**, que recoge el número de contacto directo del trabajador. Aunque este dato no es estrictamente obligatorio para formalizar el alta técnica en la TGSS, se utilizará en la plataforma para facilitar la comunicación interna y agilizar posibles trámites administrativos.

7.2.2. Datos de Domicilio

Esta sección del formulario está destinada a registrar la residencia legal del empleado, estrictamente necesaria para la emisión de notificaciones y la correcta aplicación de la normativa.

7.2.2.1. Dirección Física y Geográfica

Para conformar la dirección de manera estandarizada, se solicita el **Tipo de vía pública** (por ejemplo, CL para calle, AV para avenida, PZ para plaza), un dato exigido por la Agencia Tributaria y la TGSS para el formato oficial del domicilio. A este campo

le sigue la **Vía pública**, que recoge el nombre de la calle o avenida y forma parte del domicilio fiscal. Para detallar la ubicación exacta dentro del edificio, se deben rellenar los campos correspondientes al **Número de la calle, piso y puerta**. Finalmente, es imperativo introducir el **ID municipio**, el cual actúa como identificador del municipio del domicilio y es un requisito ineludible para las comunicaciones a Hacienda y al SEPE.

Para reforzar la confianza en que el usuario haya introducido correctamente este último dato, al lado del código postal de la dirección se muestra el nombre del municipio asociado al ID introducido, lo que permite una verificación visual inmediata y reduce la probabilidad de errores en la introducción de este dato crítico.

7.2.3. Datos de Contrato y Formación

Este bloque asienta las bases de la relación laboral entre el trabajador y la organización empleadora.

7.2.3.1. Vinculación Corporativa y Académica

En primer lugar, se requiere el **ID de la Empresa** (código CCC), un campo esencial para la segregación de datos que vincula inequívocamente al empleado con su empresa. A continuación, se establece la **Fecha de alta**, que marca el inicio de la relación laboral y es la fecha exacta que se comunica a la TGSS; esta debe estar comprendida entre el día en el que se realiza el alta y 60 días posteriores. A nivel formativo, se solicita el **ID del nivel de estudios**, que indica el nivel máximo alcanzado por el trabajador, un dato que solicita de forma obligatoria el SEPE en la comunicación de contratos a través del sistema Contrat@.

7.2.3.2. Condiciones y Datos Bancarios

El formulario requiere seleccionar el **ID del convenio**, el identificador del Convenio Colectivo aplicable, el cual queda enlazado al campo "Código Convenio" de la entidad empresa y define las condiciones salariales mínimas del puesto. Para la futura gestión de la nómina, se recoge el **IBAN** (Código Internacional de Cuenta Bancaria), que indica la cuenta bancaria donde el trabajador recibirá sus ingresos y que debe ser custodiado con alta seguridad y cifrado. Por último, y para perfiles laborales concretos, se solicita el **Número de Tarjeta de Conductor** (tarjeta de tacógrafo), siendo esto imprescindible para el cumplimiento de la normativa en el sector del transporte.

7.2.4. Constantes (Uso Futuro y Lógica de Contrato)

Además de los datos visibles y rellenados manualmente, el sistema contempla una serie de variables internas o constantes destinadas a automatizar lógicas futuras, generación de documentos y la integración transparente con los sistemas del Estado.

7.2.4.1. Parámetros Oficiales de Seguridad Social

Se establece el campo **Situación** (por defecto "01"), que determina la situación inicial en la Seguridad Social (Alta en Régimen General) y se comunica mediante el fichero TA.2/S. A su vez, se define el **Tipo de contrato** (por ejemplo, 100 para indefinido, 420 para temporal), un código oficial que categoriza la relación laboral y dictamina

posibles bonificaciones o penalizaciones por parte de la TGSS. Este se complementa con el **Subtipo de contrato**, que detalla la modalidad específica requerida por el SEPE. Asimismo, se incluye el **Colectivo del trabajador**, que recoge códigos específicos de colectivos singulares (como personas con discapacidad) necesarios para la correcta aplicación de reducciones de cuotas en la TGSS.

7.2.4.2. Condiciones Laborales y Retributivas

La plataforma registra el **Porcentaje de jornada**, un coeficiente de parcialidad (por ejemplo, 100 para jornada completa, 50 para media jornada) que afecta de forma directa y proporcional a la cotización a la Seguridad Social. A esto se le suma el **Horario**, que aloja una descripción del horario de trabajo, siendo un dato contractual obligatorio para el correcto registro de jornada. En el apartado económico, el sistema gestiona el **Importe bruto**, es decir, el salario anual o mensual bruto del trabajador que servirá como base para el cálculo de la nómina y la base de cotización, junto con el **Tipo de importe**, que especifica la periodicidad del pago (anual, mensual o por hora).

7.2.4.3. Temporalidad, Antigüedad y Periodo de Prueba

Para los contratos de duración determinada, es obligatorio registrar la **Fecha de fin de contrato** para su comunicación al SEPE, así como el **Motivo de la temporalidad** (circunstancias de la producción, sustitución, etc.), que conforma la justificación legal ineludible exigida para el cumplimiento del Real Decreto-ley 32/2021. Finalmente, se establecen como datos contractuales clave la duración del **Periodo de prueba** aplicable al inicio de la actividad, y el campo **Antigüedad**, que actuará como variable de control interno para el futuro cálculo automatizado de trienios o complementos dentro del módulo de generación de nóminas.

Capítulo 8

Pruebas

En este capítulo se describen las pruebas de seguridad que se han realizado para comprobar la robustez del sistema frente a errores de diseño o implementación, así como la validación de los objetivos planteados. Se detallan los casos de prueba, los resultados obtenidos y se analizan las posibles vulnerabilidades detectadas durante el proceso de testeo.

8.1. Pruebas de seguridad

8.1.1. Elevación de privilegios

«Descripción de la prueba de elevación de privilegios, incluyendo el método utilizado para intentar obtener acceso no autorizado a funciones o datos restringidos, los resultados obtenidos y las medidas implementadas para mitigar esta vulnerabilidad.»

La elevación de privilegios es una vulnerabilidad crítica que permite a un atacante obtener permisos más altos de los que le han sido asignados. Para probar esta vulnerabilidad, se realizaron intentos de acceso a funciones administrativas utilizando cuentas con permisos limitados.

Para el caso individual de esta aplicación, la elevación de privilegios consiste en acceder a funciones, por ejemplo, por parte de un usuario solo con permisos de RRHH, a funciones que están reservadas solamente para aquellos que posean permisos de administrador o incluso de supervisor.

La prueba consistió en iniciar sesión con una cuenta únicamente con permisos de RRHH, y tratar de registrar un nuevo usuario con permisos de administrador, lo cual no debería ser posible.

En la fase inicial de desarrollo, se detectó que esta vulnerabilidad existía, ya que no se habían implementado las reglas de seguridad necesarias para evitarlo. Estas reglas simplemente consistían en bloquear las casillas del formulario correspondientes a los permisos de administrador, RRHH y supervisor, dejándolas deshabilitadas solo para aquellos usuarios con los permisos adecuados. Tras la implementación de estas medidas de seguridad, se volvió a realizar esta prueba, dando como resultado que el sistema bloqueaba correctamente el acceso a funciones no autorizadas. El usuario intentaría hacer click en la casilla para activar dichos permisos, solo para que la

casilla no cambie su estado, permaneciendo deshabilitada y sin permitir la selección de permisos que no correspondieran al rol del usuario.

8.1.2. Path Traversal

«Descripción de la prueba de path traversal, incluyendo el método utilizado para intentar acceder a archivos o directorios no autorizados, los resultados obtenidos y las medidas implementadas para mitigar esta vulnerabilidad.» La traversing de rutas es una vulnerabilidad que permite a un atacante acceder a archivos o directorios que no deberían estar disponibles. Para probar esta vulnerabilidad, se realizaron intentos de acceso a recursos del sistema utilizando rutas relativas o absolutas no autorizadas. En este caso, como la página web no es una de tipo SPA (Single Page Application), sino una de tipo MPA (Multi Page Application), es posible que un atacante intente acceder, modificando el final de la URL, a páginas o recursos a los que no debería acceder.

En una primera prueba, en fase de desarrollo de la página, se comprobó que esto podía ser posible, ya que durante la creación de la página, nunca fue considerado ese caso y, por lo tanto, no había puestas reglas de seguridad que bloquearan dicho comportamiento.

Sin embargo, una vez detectada esta vulnerabilidad, se añadió la siguiente regla de seguridad en el frontend, que está escrita con pseudocódigo para ilustrar el concepto, pero que se implementó de forma real en FlutterFlow utilizando la lógica de programación visual que esta plataforma ofrece, y que se aplicó a todas las páginas del sistema:

```
// Ejemplo de regla de seguridad para prevenir path traversal
si (tengo acceso a la página solicitada){
    //No hago nada, dejo que el usuario acceda a la página
}
sino {
    //Redirijo al usuario al menú principal
    redirigir('/menu_principal');
}
```

Tras la aplicación de estas medidas de seguridad en todas las páginas del sistema, se volvió a relizar esta prueba, dando como resultado que el sistema bloqueaba correctamente el acceso a páginas no autorizadas, redirigiendo al usuario al menú principal o homepage. Estas reglas de seguridad también sirven para bloquear el acceso a la aplicación a un usuario que no haya iniciado sesión, ya que, al no tener ningún rol asignado, no tendría acceso a ninguna página del sistema, redirigiéndolo al menú de inicio de sesión.

Capítulo 9

Impacto del trabajo

Este capítulo está dedicado a evaluar las repercusiones y los beneficios derivados del desarrollo de este Trabajo de Fin de Grado. En la primera sección se abordará el impacto general del proyecto desde una perspectiva tecnológica, organizativa y socioeconómica, destacando cómo las decisiones de diseño tomadas han estado condicionadas por la necesidad de aportar un valor real y seguro al entorno laboral. En la segunda sección, se analizará la alineación de la plataforma desarrollada con los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 de las Naciones Unidas.

9.1. Impacto general

El desarrollo de este sistema automatizado de altas de empleados presenta un impacto potencial altamente positivo, especialmente en el contexto organizativo de las gestorías y los departamentos de Recursos Humanos. Hasta ahora, estos sectores se han visto obligados a lidiar con procesos burocráticos repetitivos y dependientes de software de escritorio anticuado (como la aplicación SILTRA) debido a las limitaciones tecnológicas del Sistema RED de la Seguridad Social. La implementación de esta plataforma aporta una modernización drástica, transformando un proceso propenso a errores humanos en un flujo digital, ágil y guiado. Al incorporar validaciones algorítmicas en tiempo real (como la verificación de identificadores DNI/NIE o el control lógico de fechas de nacimiento y alta), se reduce significativamente el riesgo de enviar datos erróneos a la administración, lo cual a su vez previene posibles sanciones administrativas para las empresas y ahorra horas de trabajo de corrección de expedientes. A lo largo del desarrollo, se han tomado diversas decisiones técnicas fundamentadas en maximizar este impacto positivo y mitigar riesgos:

- **Adopción de una arquitectura Multi-Tenant:** La decisión más crítica fue el diseño de la base de datos documental en Firestore orientada a múltiples inquilinos. Esto se hizo con la consideración innegociable de proteger el derecho a la privacidad. Al gestionar datos altamente sensibles (identificación, salarios, domicilios), la arquitectura garantiza una segregación estricta de la información, de modo que ningún usuario pueda acceder a datos de empresas para las que no está explícitamente autorizado, cumpliendo así de forma rigurosa con el RGPD.
- **Uso de tecnologías Low-Code y Backend-as-a-Service:** La elección de Flutter-Flow y Firebase se basó en el impacto de la escalabilidad y el mantenimiento.

Se democratiza el desarrollo de software corporativo permitiendo a las gestorías, que normalmente no disponen de grandes presupuestos informáticos, acceder a una infraestructura en la nube robusta, segura y fácil de mantener.

- **Desarrollo de una Mock API de transición:** Ante la imposibilidad técnica de conectarse a una API pública moderna de la Tesorería General de la Seguridad Social (TGSS), se decidió crear un sistema de validación local propio. El impacto de esta decisión es proporcionar un entorno de pruebas y una arquitectura ya preparada (Plug and Play) para que, en un futuro, la plataforma pueda engancharse directamente a una pasarela comercial u oficial de forma transparente para el usuario final.

9.2. Objetivos de Desarrollo Sostenible

El proyecto desarrollado no solo busca una mejora tecnológica, sino que también contribuye de forma directa y transversal a la consecución de varios Objetivos de Desarrollo Sostenible (ODS) impulsados por la Agenda 2030 de la ONU. Las metas más relevantes que se ven impactadas por la naturaleza de este trabajo son las siguientes:

- **ODS 8 - Trabajo decente y crecimiento económico** Este objetivo promueve el crecimiento económico sostenido, inclusivo y sostenible, el empleo pleno y productivo, y el trabajo decente. La aplicación desarrollada tiene un impacto directo en este ODS al facilitar y agilizar la formalización legal de los contratos laborales. Al simplificar la burocracia para dar de alta a los trabajadores en la Seguridad Social, se combate la economía sumergida y se fomenta la contratación regulada, garantizando que los empleados accedan a sus derechos sociales desde el primer día de actividad. Además, aumenta la productividad y la eficiencia económica de las pequeñas y medianas empresas (pymes) y despachos laborales, permitiéndoles enfocar sus recursos en tareas de mayor valor en lugar de en la burocracia repetitiva.
- **ODS 9 - Industria, innovación e infraestructura** La meta central de este ODS es construir infraestructuras resilientes, promover la industrialización inclusiva y sostenible y fomentar la innovación. El proyecto aporta una innovación tecnológica clave al sector de la administración laboral, que tradicionalmente ha sufrido un rezago tecnológico. Al introducir paradigmas de desarrollo como el *Low-Code*, las bases de datos NoSQL e integraciones en la nube (Cloud Functions), la plataforma dota a las gestorías de una infraestructura digital moderna, escalable y accesible que moderniza su tejido industrial y sus procesos diarios.
- **ODS 16 - Paz, justicia e instituciones sólidas** Este ODS aboga por crear instituciones eficaces, responsables y transparentes a todos los niveles. Al haber diseñado la aplicación con un enfoque férreo en el cumplimiento de la legalidad (integrando validaciones requeridas por el SEPE y la TGSS) y en la seguridad de la información (garantizando el Reglamento General de Protección de Datos - RGPD mediante una separación de roles y arquitectura *Multi-Tenant*), el sistema fortalece la relación de transparencia y confianza entre las empresas, los trabajadores y la Administración Pública. Asimismo, el registro histórico (colección *altas_history*) garantiza una trazabilidad y auditoría completa de los trámites, previniendo irregularidades y fomentando la responsabilidad institucional.

Capítulo 10

Resultados y conclusiones

«Breve explicación, por secciones, de los contenidos de este capítulo»

10.1. Resultados

«Resumen de resultados obtenidos en el TFG»

En este TFG se ha desarrollado una aplicación web de gestión de empresas y empleados, con un sistema de autenticación basado en Google Auth y una arquitectura multi-tenant que garantiza la segregación de datos entre diferentes organizaciones. La aplicación permite a los administradores crear nuevas empresas y gestionar sus datos, mientras que los usuarios con permisos de recursos humanos pueden gestionar a los empleados asociados a cada empresa. Además, se han implementado medidas de seguridad para proteger la información sensible y cumplir con las normativas de protección de datos.

En cuanto a los resultados específicos, se han logrado los siguientes objetivos:

- Implementación de un sistema de autenticación seguro y eficiente utilizando Google Auth, que permite a los usuarios iniciar sesión con sus credenciales de Google y garantiza la protección de sus datos de acceso.
- Desarrollo de un módulo de gestión de empresas que permite a los administradores crear nuevas empresas y gestionar sus datos, incluyendo información legal, fiscal y de cotización.
- Implementación de un sistema de control de acceso basado en roles, que permite a los usuarios con permisos de recursos humanos gestionar a los empleados asociados a cada empresa, mientras que los administradores tienen acceso a todas las funcionalidades de gestión.
- Garantía de la segregación de datos entre diferentes organizaciones mediante una arquitectura multi-tenant, que asegura que cada empresa solo pueda acceder a su propia información y no a la de otras empresas.
- Cumplimiento de las normativas de protección de datos mediante la implementación de medidas de seguridad para proteger la información sensible de los usuarios y las empresas, así como la inclusión de cláusulas de privacidad en el proceso de alta de empleados.

10.2. Conclusiones personales

«Conclusiones personales del estudiante sobre el trabajo realizado»

Este trabajo de fin de grado ha sido una experiencia enriquecedora que me ha permitido aplicar los conocimientos adquiridos a lo largo de mi formación en un proyecto real y complejo. A lo largo del desarrollo de la aplicación, he enfrentado diversos desafíos técnicos y de diseño que me han obligado a investigar, aprender nuevas tecnologías y encontrar soluciones creativas para superar los obstáculos. He aprendido a trabajar con Google Auth para implementar un sistema de autenticación seguro, así como a diseñar una arquitectura multi-tenant que garantiza la segregación de datos entre diferentes organizaciones. Además, he adquirido experiencia en la gestión de permisos y roles, lo que me ha permitido crear una aplicación que se adapta a las necesidades de diferentes tipos de usuarios dentro de una empresa.

10.3. Trabajo futuro

«Trabajo futuro que no se haya podido realizar o siguientes pasos que tomará el desarrollo realizado en este TFG» Entre las mejoras que se podrían implementar en un futuro se encuentran:

- Implementación de notificaciones automáticas por correo electrónico para eventos clave, como la creación de una nueva empresa o la asignación de un nuevo empleado a una empresa.
- Implementación de mensajes de confirmación en el frontend para acciones críticas, como la creación de una nueva empresa o la asignación de un nuevo empleado a una empresa, para mejorar la experiencia del usuario y proporcionar retroalimentación inmediata sobre el éxito de sus acciones.
- Integración del inicio de sesión en la aplicación con una cuenta de Google, utilizando el sistema de autenticación de Google Auth para simplificar el proceso de acceso y mejorar la seguridad. Aunque en la imagen de inicio de sesión aparece dicho botón, esta funcionalidad no se ha podido implementar en el desarrollo actual del TFG, entre otras razones, porque el sistema de creación de usuarios crea a la vez el perfil de autenticación y el perfil de usuario, lo que dificulta la integración con Google Auth. Sin embargo, esta mejora se considera una prioridad para el futuro desarrollo de la aplicación, ya que proporcionaría una experiencia de usuario más fluida y segura.
- Creación de un robusto menú de configuración, en el que podrían aparecer opciones como la de cambiar la contraseña, modificar el perfil de usuario, gestionar las empresas a las que tiene acceso, etc. Este menú de configuración se podría implementar como un componente adicional en el frontend, accesible desde la página de inicio o desde cualquier otra sección de la aplicación, y permitiría a los usuarios personalizar su experiencia y gestionar sus datos de manera más eficiente.
- Implementación de todas las lenguas cooficiales de España, para hacer la aplicación más accesible a un público más amplio. Esto implicaría la traducción de todos los textos y mensajes de la aplicación a las diferentes lenguas, así como la adaptación de la interfaz de usuario para soportar caracteres especiales y

formatos de fecha y hora específicos de cada lengua. Sería una buena práctica teniendo en cuenta que el marco legal y laboral en el que se enmarca esta aplicación es el español, y que muchas de las empresas que podrían beneficiarse de esta aplicación podrían estar ubicadas en regiones con lenguas cooficiales. También facilitaría la adopción de la aplicación por parte de empresas y usuarios que prefieren utilizar su lengua materna, mejorando así la experiencia del usuario y aumentando la satisfacción con la aplicación.

Capítulo 11

Anexo

11.1. Estructuras de dades

11.1.1. Lista de datos de una empresa corporativa

A continuación, se muestra una lista con todos los campos que se almacenan en el documento de cada organización o empresa cliente dentro de la base de datos Firestore, detallando el tipo de dato y su funcionalidad en el sistema:

- **ID de la Empresa (String):** Clave única del documento que identifica a la entidad dentro del sistema, actuando como pilar fundamental para la segregación de datos en la arquitectura Multi-Tenant.
- **Razón Social (String):** Nombre legal completo de la organización corporativa utilizado de forma obligatoria en la elaboración de contratos, nóminas y trámites oficiales con el SEPE.
- **CIF (String):** Código de Identificación Fiscal de la entidad, estrictamente necesario para las comunicaciones, validaciones y trámites ante la Agencia Tributaria.
- **Domicilio Social (String):** Dirección legal completa de la sede de la empresa empleada para la recepción de notificaciones formales.
- **ID del Municipio (String):** Código identificador del municipio del domicilio social, utilizado para la ubicación geográfica requerida en los trámites con el SEPE.
- **ID del Municipio de cotización (String):** Identificador geográfico de uso exclusivo por parte de la Tesorería General de la Seguridad Social (TGSS) para gestiones de afiliación; su asignación se realiza de forma automática en el backend mediante vinculación relacional con el domicilio social.
- **Régimen (String):** Régimen de la Seguridad Social aplicable a la actividad laboral de la empresa (establecido como el Régimen General para la mayoría de las organizaciones).
- **CCC (String):** Código de Cuenta de Cotización. Clave alfanumérica única ante la Seguridad Social que asocia el centro de trabajo y la mutua, constituyendo la llave indispensable para procesar cualquier alta técnica.
- **Código de actividad económica o CNAE (String):** Código numérico oficial de

clasificación de la actividad comercial de la empresa exigido obligatoriamente en el registro.

- **Actividad económica (String):** Descripción literal y detallada de la labor comercial de la entidad basada directamente en la clasificación del CNAE.
- **Clave percepto (String):** Identificador tributario empleado por la Agencia Tributaria para clasificar la naturaleza de los ingresos o pagos y proceder al cálculo de retenciones.
- **Administración de Hacienda (String):** Identificación de la delegación de Hacienda a la cual se encuentra adscrita la sede fiscal para la correcta canalización documental.
- **Código Idioma MEC (String):** Variable de control para regular normativamente el idioma empleado en las comunicaciones con el Ministerio de Empleo y Seguridad Social.
- **Código convenio (String):** Código unívoco del convenio colectivo que aplica a la organización, imprescindible para la clasificación del personal ante la TGSS.
- **Convenio colectivo (String):** Denominación oficial y textual del convenio regulador, cuya presencia es de obligado cumplimiento en la confección física o digital del contrato.
- **Conceptos salariales (Map / Estructural):** Estructura de datos que delimita los componentes base de las nóminas (sueldo base, complementos y pagas extraordinarias), sirviendo como punto de partida para interactuar con futuros módulos de nóminas.
- **NIF Firmante (String):** Documento de identidad de la persona con poderes legales o autorizada para rubricar los contratos de trabajo y tramitar las comunicaciones del sistema Contrat@ del SEPE.
- **Concepto Firmante (String):** Cargo o condición representativa del firmante autorizado (por ejemplo, “Apoderado” o “Gerente”) para su inclusión formal en el contrato.
- **Firma Copia básica (String):** Ruta de almacenamiento o referencia a la imagen digitalizada de la rúbrica del responsable, empleada en procesos automáticos de generación de contratos para la Representación Legal de los Trabajadores (RLT).
- **Formato Carta (String):** Plantilla predefinida parametrizada en la plataforma para agilizar el flujo interno de la gestoría en la emisión de notificaciones rápidas.
- **Ruta archivo (String):** Directorio del sistema configurado para el alojamiento estructurado y trazable de los archivos maestros de la empresa, como los modelos de contrato.
- **ID de cláusula (String):** Identificador que vincula el documento de la empresa con sus cláusulas de Protección de Datos específicas, garantizando que el formulario de alta incluya los textos obligatorios del art. 13 del RGPD.

11.1.2. Lista de datos de un empleado

A continuación, se detalla la estructura completa de los campos recopilados en el formulario de alta y edición de trabajadores, los cuales se almacenan en la colección de empleados vinculada de forma unívoca a la empresa correspondiente:

- **Tipo de Documento (String):** Selección oficial del documento identificativo aportado por el trabajador (DNI, NIE o Pasaporte).
- **DNI/NIE (String):** Clave principal de identificación fiscal y laboral del empleado; su formato queda condicionado por el tipo de documento seleccionado.
- **NAF (String):** Número de Afiliación a la Seguridad Social. Identificador único e ineludible del trabajador ante la TGSS, obligatorio para tramitar y validar la afiliación.
- **Nombre (String):** Nombre de pila del trabajador para su identificación formal in contracts, comunicaciones y nóminas.
- **Apellido1 (String):** Primer apellido del empleado para el registro legal formal.
- **Apellido2 (String):** Segundo apellido del empleado para completar la filiación reglamentaria.
- **Sexo (String):** Género del trabajador, requerido para la cumplimentación del Modelo 145 del IRPF y fines estadísticos de la Seguridad Social.
- **Fecha de nacimiento (Timestamp):** Fecha de nacimiento del empleado, dato estrictamente obligatorio para validar su identidad y mayoría de edad laboral ante la TGSS.
- **ID País (String):** Código identificador del país de nacionalidad, clave para discernir administrativamente entre personal comunitario y extracomunitario.
- **Teléfono (String):** Número telefónico de contacto directo. No es obligatorio para el alta técnica en el Estado, pero se guarda en backend para optimizar la comunicación interna de la plataforma.
- **Tipo de vía pública (String):** Sigla normalizada de la vía (CL para calle, AV para avenida, etc.) exigida para el formato del domicilio oficial por la AEAT y la TGSS.
- **Vía pública (String):** Nombre textual de la calle, avenida o plaza que conforma la residencia fiscal del trabajador.
- **Número (String/Integer):** Número físico del inmueble del domicilio del trabajador.
- **Piso (String):** Planta o piso específico de la vivienda del empleado.
- **Puerta (String):** Puerta o letra identificativa del domicilio del empleado.
- **ID municipio (String):** Código identificador oficial del municipio de residencia, obligatorio para los envíos documentales hacia Hacienda y el SEPE.
- **ID de la Empresa / CCC (String):** Código de cuenta de cotización de la entidad contratante que vincula estructuralmente al empleado con su organización en Firestore.

- **Fecha de alta (Timestamp):** Fecha exacta del inicio de los efectos de la relación laboral que se transmite a la TGSS; debe estar en un rango de entre el día actual y un máximo de 60 días posteriores.
- **ID del nivel de estudios (String):** Código del nivel máximo de estudios completados por el trabajador, solicitado de forma obligatoria por el SEPE en el sistema Contrat@.
- **ID del convenio (String):** Identificador del convenio colectivo aplicable que asocia los baremos salariales mínimos permitidos al puesto.
- **IBAN (String):** Código Internacional de Cuenta Bancaria para la posterior transferencia de haberes; este dato operacional sensible es custodiado bajo estrictos mecanismos de cifrado.
- **Número de Tarjeta de Conductor (String):** Código identificador de la tarjeta de tacógrafo, requerido de forma exclusiva para el cumplimiento normativo en el sector del transporte.
- **Situación (String) [Constante]:** Variable interna fijada por defecto en “01” (Alta en el Régimen General) para la estructuración automática del fichero oficial TA.2/S.
- **Tipo de contrato (String) [Constante]:** Código numérico oficial (v.g., 100 para indefinido, 420 para temporal) que categoriza el tipo de contrato e influye en las bonificaciones o recargos de la TGSS.
- **Subtipo de contrato (String) [Constante]:** Código de desglose que detalla la modalidad específica contractual para el SEPE.
- **Colectivo del trabajador (String) [Constante]:** Código interno utilizado para reflejar colectivos singulares (v.g., personas con discapacidad) necesarios para la aplicación de reducciones de cuotas.
- **Porcentaje de jornada (Number) [Constante]:** Coeficiente de parcialidad de la prestación laboral (100 para jornada completa, 50 para media jornada) que incide directamente en las bases de cotización.
- **Horario (String) [Constante]:** Descripción textual de la distribución del tiempo de trabajo, obligatoria según el registro normativo de jornada.
- **Importe bruto (Number) [Constante]:** Retribución económica bruta asignada al empleado, que sirve como base aritmética para la simulación de nóminas y bases de cotización.
- **Tipo de importe (String) [Constante]:** Indicador de la periodicidad de la cuantía salarial introducida (anual, mensual o por horas).
- **Fecha de fin de contrato (Timestamp) [Constante]:** Fecha de término de la relación laboral en contratos de naturaleza temporal, obligatoria para el SEPE.
- **Motivo de la temporalidad (String) [Constante]:** Causa legal justificativa de la duración determinada del contrato según las exigencias del RD-ley 32/2021.
- **Periodo de prueba (String) [Constante]:** Duración pactada por contrato del periodo de prueba al inicio de la actividad laboral.

- **Antigüedad (Timestamp) [Constante]:** Fecha de control interno de antigüedad del empleado, reservada para la futura automatización de trienios y pluses salariales.

11.1.3. Lista de datos de un convenio

A continuación, se muestra una lista con todos los campos que se almacenan en el documento de cada convenio dentro de la base de datos Firestore, incluyendo el tipo de dato y su función dentro del sistema de gestión de contratación y relaciones laborales:

- **Actividad_Economica (String):** Descripción textual de la actividad comercial principal regulada por el convenio (por ejemplo, "TRANSPORTE MERCANCIAS POR CARR").
- **CNO (String):** Código de la Clasificación Nacional de Ocupaciones que identifica la categoría estadística oficial del puesto.
- **Categoria (String):** Identificador abreviado de la categoría profesional del trabajador dentro de la estructura organizativa (por ejemplo, "COND.MEC").
- **Cobro (String):** Especifica la periodicidad del pago o devengo salarial establecido para el puesto (por ejemplo, "Mensual").
- **Codigo_Actividad_Economica (String):** Código numérico que clasifica el sector de actividad económica del convenio.
- **Codigo_OcupacionSS (String):** Clave oficial de ocupación exigida por la Seguridad Social para el cálculo de los tipos de cotización por Accidentes de Trabajo y Enfermedades Profesionales (AT y EP).
- **Convenio_Colectivo (String):** Código unívoco oficial de registro del convenio colectivo ante el Ministerio de Trabajo.
- **Convenio_Navision (String):** Identificador o código de correspondencia interna utilizado para la integración y sincronización de datos con el software ERP Navision.
- **Convenio_Puesto (String):** Código compuesto estructural que vincula el código del convenio con la categoría profesional específica para su gestión en el sistema.
- **Denominacion_Completa (String):** Nombre legal completo y descriptivo del convenio colectivo regulador, incluyendo su ámbito geográfico o provincial.
- **Empresa (String):** Nombre o razón social de la empresa asociada a la aplicación de este documento normativo.
- **Grupo_Cotizacion (String):** Código de dos dígitos que determina el grupo de cotización de la Seguridad Social asignado al puesto (por ejemplo, "08"), clave para fijar las bases mínimas y máximas.
- **IdConvenio (Int64):** Identificador numérico único de control interno del documento de convenio dentro del sistema.
- **IdEmpresa (Int64):** Identificador numérico que vincula relacionalmente este convenio con el documento único de la empresa correspondiente.

- **Nombre_Provincia (String):** Denominación textual de la provincia a la que se circunscribe el ámbito de aplicación del convenio colectivo.
- **Original (String):** Listado o registro de texto de las entidades y empresas firmantes u originarias ligadas inicialmente a este marco normativo.
- **Provincia (String):** Código de dos dígitos asignado formalmente por el Instituto Nacional de Estadística (INE) a la provincia reguladora (por ejemplo, “30” para Murcia).
- **Puesto_Trabajo (String):** Denominación formal del puesto laboral que desempeña el operario dentro de la organización (por ejemplo, “CONDUCTOR”).
- **Regimen (String):** Código numérico de cuatro dígitos oficial correspondiente al régimen de la Seguridad Social aplicable para la cotización.
- **Regimen_Navision (String):** Nombre descriptivo del régimen de cotización procesado de forma interna por el software de gestión Navision (por ejemplo, “GENERAL”).
- **Trabajador_TMS (String):** Identificador o etiqueta del perfil profesional utilizado para la interconexión con el Sistema de Gestión de Transportes (TMS).

11.1.4. Lista de datos de un municipio

A continuación, se muestra una lista con todos los campos que se almacenan en el documento de cada municipio dentro de la base de datos Firestore, detallando el tipo de dato y su función para la validación geográfica y tramitación de altas en el sistema:

- **codigo_postal (String):** El código postal de cinco dígitos que identifica la zona de reparto e influye en la clasificación censal del centro de trabajo o del domicilio del empleado.
- **municipio_id (String):** Identificador numérico o alfanumérico único oficial asignado al municipio, utilizado como clave de correspondencia obligatoria en las comunicaciones enviadas hacia Hacienda, el SEPE y la Seguridad Social.
- **nombre_municipio (String):** Denominación textual completa y literal del municipio (por ejemplo, “Ponteareas”), empleada para proporcionar una verificación visual inmediata en la interfaz y mitigar la introducción errónea de datos críticos.

11.1.5. Lista de datos de un nivel de estudios

A continuación, se muestra una lista con todos los campos que se almacenan en el documento de cada nivel de estudios dentro de la base de datos Firestore, detallando el tipo de dato y su finalidad para la correcta codificación académica y contractual exigida por el SEPE:

- **Codigo (String):** Código numérico de dos dígitos oficial que identifica el nivel educativo alcanzado por el trabajador, indispensable para la cumplimentación de contratos en el sistema Contrat@.

- **Titulo (String):** Descripción literal completa y reglamentaria de la titulación o la formación correspondiente al código (por ejemplo, “Enseñanzas de grado medio de formación profesional específica, artes plásticas y diseño y deportivas”).

11.2. Funciones de Dart utilizadas en el desarrollo

11.2.1. Funciones de validación de campos

11.2.1.1. Validación de DNI/NIE

Listing 11.1: Función de validación de DNI/NIE

```
1 bool validarDniNie(  
2   String documento,  
3   String tipoDocumento,  
4 )  
5 {  
6   const String tablaLetras = 'TRWAGMYFPDXBNJZSQVHLCKE';  
7   final String docUpper = documento.toUpperCase();  
8  
9  
10  if (docUpper.length != 9) {  
11    return false;  
12  }  
13  
14  
15  
16  final RegExp dniNieRegex =  
17    RegExp(r'^(?:[0-9]{8}|[XYZ][0-9]{7})[TRWAGMYFPDXBNJZSQVHLCKE]\$');  
18  
19  if (!dniNieRegex.hasMatch(docUpper)) {  
20    return false;  
21  }  
22  
23  
24  final String letraDocumento = docUpper[8];  
25  String parteNumericaStr = docUpper.substring(0, 8);  
26  
27  String primeraLetra = parteNumericaStr[0];  
28  
29  if (primeraLetra == 'X' || primeraLetra == 'Y' || primeraLetra == 'Z') {  
30    String digitoReemplazo;  
31    switch (primeraLetra) {  
32      case 'X':  
33        digitoReemplazo = '0';  
34        break;  
35      case 'Y':  
36        digitoReemplazo = '1';  
37        break;  
38      case 'Z':  
39        digitoReemplazo = '2';  
40        break;  
41      default:  
42        return false;  
43    }  
44    parteNumericaStr = digitoReemplazo + parteNumericaStr.substring(1);  
45  }  
46  final int parteNumerica = int.tryParse(parteNumericaStr) ?? 0;  
47  final int resto = parteNumerica % 23;  
48  final String letraCalculada = tablaLetras[resto];  
49  return letraCalculada == letraDocumento;  
50 }
```

11.2.1.2. Limitación de la edad a un mínimo de 16 años

```
1 DateTime getTopBornTime() {
2   /// MODIFY CODE ONLY BELOW THIS LINE
3
4   const int edadMinima = 16;
5
6
7   final hoy = DateTime.now();
8
9   final fechaCorte = DateTime(
10     hoy.year - edadMinima,
11     hoy.month,
12     hoy.day
13   );
14
15   return fechaCorte;
16 }
```

11.2.1.3. Limitación de la fecha de alta a un máximo de 60 días posteriores a la fecha actual

Listing 11.2: Función para obtener el máximo día de alta

```
1 DateTime obtenerMaximoDiaAlta() {
2   /// MODIFY CODE ONLY BELOW THIS LINE
3   int diasASumar = 60;
4   final hoy = DateTime.now();
5   final duracion = Duration(days: diasASumar);
6   final fechaCorte = hoy.add(duracion);
7   return fechaCorte;
8 }
```

11.2.1.4. Validar NASS

Listing 11.3: Función de validación de NAF

```
1 bool validarNASS(String nass) {
2   /// MODIFY CODE ONLY BELOW THIS LINE
3
4
5   final RegExp nassRegExp = RegExp(r'^[0-9]{12}\\$');
6   if (!nassRegExp.hasMatch(nass)) {
7     return false;
8   }
9
10
11   final int codProv = int.parse(nass.substring(0, 2));
12   final int numAfiliado = int.parse(nass.substring(2, 10));
13   final int control = int.parse(nass.substring(10, 12));
14   final int numGrande = int.parse(nass.substring(0, 10));
15
16
17   int comprobacion = (numAfiliado < 10000000)
18     ? (numAfiliado + codProv * 10000000) % 97
19     : numGrande % 97;
20
21   return comprobacion == control;
22 }
```

11.2.1.5. Enviar alta

Listing 11.4: Función de envío de alta a la TGSS

```
1 Future<String> submitAlta(  
2   String tipoDocumento,  
3   String dniNie,  
4   String nass,  
5   String prefijoTlf,  
6   String numTlf,  
7   DateTime fechaAlta,  
8   DateTime? fechaBaja,  
9 ) async {  
10  
11   if (!validarDniNie(dniNie, tipoDocumento)) {  
12     return "ERROR: el \${tipoDocumento} no es valido";  
13   }  
14  
15   else if (!validarNASS(nass)) {  
16     return "ERROR: el NASS no es valido";  
17   }  
18  
19   else if (!isFechaBajaCorrect(fechaAlta, fechaBaja)) {  
20     return "ERROR: la fecha de baja no puede ser anterior a la fecha de alta";  
21   }  
22  
23  
24   return "OK";  
25 }
```

11.2.1.6. Obtener referencias de documentos a partir de sus IDs

Listing 11.5: Función para obtener referencias de documentos a partir de sus IDs

```
1 import 'package:cloud_firestore/cloud_firestore.dart';  
2  
3 Future<List<DocumentReference>> getDocRefFromID(  
4   List<String>? docIDs, String collection) async {  
5   // Add your function code here!  
6   final FirebaseFirestore firestore = FirebaseFirestore.instance;  
7   final List<DocumentReference> documentReferences = [];  
8  
9   // Iterar sobre la lista de IDs de documentos  
10  for (String docID in docIDs!) {  
11    // 1. Construir la ruta del documento  
12    String documentPath = "\${collection}/${docID}";  
13  
14    // 2. Crear la referencia al documento (operacion local)  
15    DocumentReference documentReference = firestore.doc(documentPath);  
16  
17    // 3. Poner la referencia en la lista de resultados  
18    documentReferences.add(documentReference);  
19  }  
20  
21  // 4. Devolver la lista de referencias  
22  return documentReferences;  
23 }
```

11.2.1.7. Obtener referencia de un usuario

Listing 11.6: Función para obtener la referencia de un usuario a partir de su ID

```
1 import 'package:cloud_firestore/cloud_firestore.dart';  
2  
3 Future<DocumentReference> getSingleUserRefFromID(String docID) async {  
4   final FirebaseFirestore firestore = FirebaseFirestore.instance;  
5 }
```

11.2. Funciones de Dart utilizadas en el desarrollo

```
6 String documentPath = "/users/" + "\$docID"; //collection you want to query
7 DocumentReference documentReference = firestore.doc(documentPath);
8 return documentReference;
9 }
```

11.2.1.8. Crear un nuevo usuario en la colección de usuarios

Listing 11.7: Función para crear un nuevo usuario

```
1 import 'package:firebase_auth/firebase_auth.dart';
2 import 'package:firebase_core/firebase_core.dart';
3
4 Future<String> manageUser(
5     // Parameters
6     String emailAddress, //Email from Widget State
7     String password, //Password from Widget State
8     String randomDocGen, //Random String (length: 28 - upper/lowe/digits)
9     bool isAdmin,
10    bool isSupervisor,
11    bool isRrhh,
12    String displayName,
13    String phoneNumber,
14    List<String> empresasRef,
15    String rol,
16    String departamento) async
17 {
18     String returnmsg = 'Success';
19     DateTime createTime = DateTime.now();
20     List<DocumentReference>? referencias = await getCompaniesRefID(empresasRef);
21     // Create a secondary Firebase app to isolate the user creation process
22     FirebaseApp app = await Firebase.initializeApp(
23         name: randomDocGen,
24         options: Firebase.app().options,
25     );
26
27     try {
28         // Create the user with the provided email and password
29         UserCredential userCredential =
30             await FirebaseAuth.instanceFor(app: app).createUserWithEmailAndPassword(
31                 email: emailAddress,
32                 password: password,
33             );
34
35         // Retrieve the UID of the newly created user
36         String? uid = userCredential.user?.uid;
37
38         if (uid != null) {
39             // Reference to the Firestore users collection
40
41             final CollectionReference usersRef =
42                 FirebaseFirestore.instance.collection('users');
43
44             // Create a new user document with the provided data
45             // 'empresas_autorizadas_ref': getCompaniesRefID(empresasRef),
46             await usersRef.doc(uid).set({
47                 'uid': uid,
48                 'email': emailAddress,
49                 'created_time': createTime,
50                 'is_admin': isAdmin,
51                 'is_supervisor': isSupervisor,
52                 'is_rrhh': isRrhh,
53                 'display_name': displayName,
54                 'phone_number': phoneNumber,
55                 'empresas_autorizadas_ref': referencias,
56                 'visibility': true,
57                 'rol': rol,
58                 'departamento': departamento,
```

```
59     });
60   } else {
61     return "Error while creating the UID";
62   }
63 } on FirebaseAuthException catch (e) {
64   return e.code;
65 } finally {
66   // Delete the secondary app to clean up resources
67
68   await app.delete();
69 }
70
71 return returnmsg;
72 }
```

11.2.2. Devolver una referencia de usuario concreta

Listing 11.8: Función para obtener la referencia de un usuario a partir de su ID

```
1 import 'package:cloud_firestore/cloud_firestore.dart';
2
3 Future<DocumentReference> getSingleCompanyRefFromID(String docID) async {
4   final FirebaseFirestore firestore = FirebaseFirestore.instance;
5
6   String documentPath = "/companies/" + "\$docID"; //collection you want to query
7   DocumentReference documentReference = firestore.doc(documentPath);
8   return documentReference;
9 }
10 // Set your action name, define your arguments and return parameter,
11 // and then add the boilerplate code using the green button on the right!
```

11.2.2.1. Obtener múltiples referencias de empresas

Listing 11.9: Función para obtener referencias de empresas a partir de sus IDs

```
1 import 'package:cloud_firestore/cloud_firestore.dart';
2
3 Future<List<DocumentReference>?> getCompaniesRefID(List<String>? docIDs) async {
4   // Add your function code here!
5   final FirebaseFirestore firestore = FirebaseFirestore.instance;
6   final List<DocumentReference> documentReferences = [];
7
8   // Iterar sobre la lista de IDs de documentos
9   for (String docID in docIDs!) {
10     // 1. Construir la ruta del documento
11     String documentPath = "/companies/\$docID";
12
13     // 2. Crear la referencia al documento (operacion local)
14     DocumentReference documentReference = firestore.doc(documentPath);
15
16     // 3. Poner la referencia en la lista de resultados
17     documentReferences.add(documentReference);
18   }
19
20   // 4. Devolver la lista de referencias
21   return documentReferences;
22 }
23
24 // Set your action name, define your arguments and return parameter,
25 // and then add the boilerplate code using the green button on the right!
```

11.2.2.2. Enviar los datos de empleados a los endpoints de la empresa

Listing 11.10: Función para enviar los datos de empleados a los endpoints de la empresa

```
1 import 'dart:convert';
2 import 'package:http/http.dart' as http;
3
4 /// Envia los datos completos de un trabajador a una API REST en formato JSON.
5 ///
6 /// Retorna true si la solicitud fue exitosa (codigo 200 o 201), false en caso contrario.
7 Future<bool> sendWorkerData(
8   String apiUrl,
9   // Datos de Contacto
10  String tipoDocumento,
11  String dniNie,
12  String naf,
13  String nombre,
14  String primerApellido,
15  String segundoApellido,
16  String email,
17  String sexo,
18  String telefono,
19  String fechaNacimiento,
20  String siglasPaisNacimiento,
21  // Datos de Domicilio
22  String numero,
23  String piso,
24  String puerta,
25  String codigoPostal,
26  String nombreMunicipio,
27  String identificadorMunicipio,
28  // Datos de Contrato y Formacion
29  String fechaAlta,
30  String? fechaBaja, // Opcional, por defecto String vacio
31  String iban,
32  String? numeroTarjetaConductor, // Opcional, por defecto String vacio
33  String codigoDelConvenio,
34 ) async {
35   // 1. Construir la estructura de datos anidada (Mapa de Dart)
36   final Map<String, dynamic> dataToSend = {
37     "datos_contacto": {
38       "tipo_documento": tipoDocumento,
39       "dni_nie": dniNie,
40       "naf": naf,
41       "nombre": nombre,
42       "primer_apellido": primerApellido,
43       "segundo_apellido": segundoApellido,
44       "email": email,
45       "sexo": sexo,
46       "telefono": telefono,
47       "fecha_nacimiento": fechaNacimiento,
48       "siglas_pais_nacimiento": siglasPaisNacimiento,
49     },
50     "datos_domicilio": {
51       "numero": numero,
52       "piso": piso,
53       "puerta": puerta,
54       "codigo_postal": codigoPostal,
55       "nombre_municipio": nombreMunicipio,
56       "identificador_municipio": identificadorMunicipio,
57     },
58     "datos_contrato_y_formacion": {
59       "fecha_alta": fechaAlta,
60       // Logica para enviar null si la fecha de baja esta vacia
61       "fecha_baja": fechaBaja!.isEmpty ? null : fechaBaja,
62       "iban": iban,
63       // Logica para enviar null si el numero de tarjeta esta vacio
64       "numero_tarjeta_conductor":
65         numeroTarjetaConductor!.isEmpty ? null : numeroTarjetaConductor,
66       "codigo_del_convenio": codigoDelConvenio,
67     }
68   };
69   // 2. Enviar los datos a la API REST
70   http.Response response = http.post(
71     apiUrl,
72     headers: {'Content-Type': 'application/json'},
73     body: jsonEncode(dataToSend));
74   return response.statusCode == 200 || response.statusCode == 201;
75 }
```

```
69
70 // 2. Codificacion JSON (Serializacion)
71 final String jsonBody = jsonEncode(dataToSend);
72
73 try {
74     // 3. Realizacion de la Solicitud HTTP (POST)
75     final response = await http.post(
76         Uri.parse(apiUrl),
77         // CRUCIAL: Indicamos a la API que el cuerpo es JSON
78         headers: <String, String>{
79             'Content-Type': 'application/json; charset=UTF-8',
80         },
81         // El cuerpo debe ser la cadena JSON codificada
82         body: jsonBody,
83     );
84
85     // 4. Manejo de la Respuesta
86     if (response.statusCode == 200 || response.statusCode == 201) {
87         print(
88             "Trabajador registrado exitosamente. Status: \${response.statusCode}");
89         // Opcional: Procesar la respuesta de la API si devuelve datos
90         // print("Respuesta de la API: \${response.body}");
91         return true;
92     } else {
93         print("Error al registrar trabajador.Codigo: \${response.statusCode}");
94         print("Cuerpo de error: \${response.body}");
95         return false;
96     }
97 } catch (e) {
98     print("Excepcion al enviar datos: \${e}");
99     return false;
100 }
101 }
102
103 // Set your action name, define your arguments and return parameter,
104 // and then add the boilerplate code using the green button on the right!
```